

دانشگاه زنجان

دانشکده مهندسی

پایان نامه دوره کارشناسی

گروه مهندسی کامپیوتر

طراحی و پیاده سازی سامانه جست و جو و ثبت آگهی بیلبورد

نگارش

علیرضا زارعیان

استاد راهنما

دکتر سجاد حقزاد کلیدبری

شهریور ۱۴۰۵

تعهدنامه اصالت اثر

اینجانب **علیرضا زارعیان** متعهد می‌شوم که مطالب مندرج در این پایان‌نامه با عنوان «طراحی و پیاده‌سازی سامانه جست‌وجو و ثبت آگهی بیلبورد» حاصل کار پژوهشی اینجانب است و به دستاوردهای پژوهشی دیگران که در این پژوهش از آن‌ها استفاده شده است، مطابق مقررات ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نشده است.

نام و نام خانوادگی دانشجو: **علیرضا زارعیان**

امضا:

تاریخ: شهریور ۱۴۰۵

فرم افشای میزان و نحوه استفاده از هوش مصنوعی

در تهیه این پروژه از دستیار هوش مصنوعی استفاده شده است. جزئیات به شرح زیر اعلام می‌شود:

حوزه	میزان و نحوه استفاده
ابزار مورد استفاده	دستیار برنامه‌نویسی مبتنی بر مدل زبانی بزرگ، در محیط خط فرمان و ویرایشگر کد.
نگارش کد	بخش قابل توجهی از کد با کمک دستیار نوشته شده است. نیازمندی‌ها، معماری، تصمیم‌های فنی و پذیرش یا رد هر پیشنهاد بر عهده نگارنده بوده است. برای جلوگیری از انحراف دستیار از قواعد پروژه، دو سند قید و شرط (CLAUDE.md و AGENTS.md) در مخزن نوشته شده که ده قاعده غیرقابل نقض پروژه را در هر جلسه به دستیار تحمیل می‌کند.
نگارش متن پایان‌نامه	ماده خام همه فصل‌ها مستندات فنی خود پروژه است که در طول توسعه نوشته شده‌اند. نگارنده متن را نوشته و ساختار داده است؛ کمک دستیار به ویرایش و یکدست‌سازی محدود بوده است. هیچ بخشی از این متن ترجمه ماشینی یک منبع دیگر نیست و هیچ مرجعی بدون مطالعه ارجاع داده نشده است.
اعداد و ادعای متن	تمام آمار این سند در زمان نگارش با اجرای مستقیم روی پروژه اندازه‌گیری شده است: شمارش رکوردها با پرس‌وجوی SQLite ، شمارش مسیرها و خطوط کد با پیمایش مخزن، و نتیجه آزمون‌ها با اجرای npm test . روش هر اندازه‌گیری در فصل ۶ آمده است.

موضع نگارنده

این دستیار نقش یک ابزار پرسرعت را داشته است، نه نقش تصمیم‌گیرنده. آنچه ارزش مهندسی این کار را می‌سازد – انتخاب معماری، تشخیص اینکه کدام قابلیت ساخته نشود، و پیدا کردن خطاهایی که خود ابزار تولید کرده بود – در فصل‌های ۴ تا ۶ مستند شده است. نمونه روشنش در بخش ۲-۶ آمده: یک کلاس خطا که مکرراً تکرار می‌شد، شناسایی و به یک قاعده دائمی در AGENTS.md تبدیل شد تا دوباره پیش نیاید.

نام و نام خانوادگی دانشجو: علیرضا زارعیان

امضا:

تاریخ: شهریور ۱۴۰۵

چکیده

بازار رسانه‌های تبلیغاتی محیطی (بیلبرد، پل عابر، استرابورد و نمایشگر شهری) در ایران بازاری پراکنده است. اطلاعات هر رسانه نزد مالک یا پیمانکار همان رسانه است و خریدار تبلیغات برای یافتن گزینه مناسب باید جداگانه با چند واسطه تماس بگیرد. نتیجه، نبود یک نمای واحد و قابل جست‌وجو از عرضه موجود است.

در این پایان‌نامه یک سامانه تحت وب طراحی و پیاده‌سازی شده که این عرضه را در یک فهرست یکپارچه قابل جست‌وجو گرد می‌آورد. سامانه دو گروه کاربر دارد: خریدار تبلیغات که میان ۳,۵۳۶ رسانه در ۱۰۱ شهر جست‌وجو و فیلتر می‌کند، جزئیات و موقعیت هر رسانه را روی نقشه می‌بیند و پس از احراز هویت شماره تماس مالک را دریافت می‌کند؛ و صاحب رسانه که آگهی خود را از راه یک فرم چندمرحله‌ای ثبت می‌کند تا پس از بررسی کارشناس منتشر شود.

سامانه با 16 Next.js و 19 React در یک پایگاه کد یکپارچه پیاده‌سازی شده است؛ لایه داده روی SQLite از طریق Prisma 7، و احراز هویت با توکن امضا شده در کوکی HttpOnly. رابط کاربری کاملاً فارسی و راست‌به‌چپ است. داده اولیه با یک خزنده Python از سه منبع عمومی گردآوری و پس از حذف تکراری‌ها یکپارچه شده است.

تمرکز اصلی کار بر کیفیت مهندسی سمت سرور بوده است. همه دسترسی به پایگاه داده از یک لایه واحد می‌گذرد و ابزار بررسی ایستای کد، هر دسترسی دیگری را رد می‌کند؛ هر مسیر برنامه‌نویسی از یک زنجیره بررسی ثابت می‌گذرد که یک بار نوشته شده و ترتیب مرحله‌هایش عوض نمی‌شود؛ هر ورودی کاربر پیش از رسیدن به پایگاه داده در برابر یک طرح‌واره سنجیده می‌شود؛ سقف نرخ درخواست به شناسه‌ای بسته شده که کاربر نمی‌تواند جعلش کند؛ اگر کاربر یک درخواست را دو بار بفرستد اثرش تنها یک بار اعمال می‌شود؛ و هر تغییر مدیریتی در یک دفتر تغییرناپذیر ثبت می‌گردد. صحت سامانه با ۱۷۹ آزمون خودکار – ۱۲ آزمون واحد روی قاعده‌های محصول و ۱۶۷ آزمون که هر یک با یک درخواست واقعی HTTP یا یک اجرای واقعی واردکننده داده کار می‌کند – و ۱۰ جریان کاربری در مرورگر واقعی سنجیده شده و همگی روی یک نسخه ساخته شده (تولیدی) با موفقیت اجرا شده‌اند.

وضعیت پروژه در زمان نگارش

این سامانه یک نمونه کارآمد و در حال اجرا است، نه یک محصول مستقرشده. تا این تاریخ روی هیچ دامنه عمومی مستقر نشده و روی رایانه توسعه اجرا می‌شود. بخش‌هایی از آن عمداً ساخته نشده‌اند (رزرو آتلاین، درگاه پرداخت، به‌روزرسانی خودکار داده) و دلیل هر یک در متن آمده است. آنچه ساخته شده کار می‌کند و آزموده شده؛ آنچه نشده، فهرست و توجیه شده است. مسیر تکمیل آن نیز روشن و بدون نیاز به بازطراحی است (فصل ۷).

یادداشت نگارنده

این نخستین وب‌سایتی است که من، علیرضا زارعیان، در تمام عمرم ساخته‌ام – نخستین بار است که یک سامانه تحت وب را از صفر طراحی و پیاده‌سازی می‌کنم. تلاش کرده‌ام کیفیت مهندسی را در حد توانم بالا نگه دارم و پشت هر تصمیم یک استدلال مستند بگذارم؛ با این حال طبیعی است که به دلیل همین بی‌تجربگی نخستین پروژه، جای‌جای کار ایرادهای ساختاری کوچکی داشته باشد. این نوشته با همین صداقت تقدیم می‌شود.

کلیدواژه: تبلیغات محیطی، بازارگاه تحت وب، Next.js، رندر سمت سرور، امنیت واسط برنامه‌نویسی، همروندی، SQLite

مسئولیت اجتماعی و اثرگذاری پژوهش

این سامانه اطلاعاتی را که امروز پراکنده و در اختیار تعداد کمی واسطه است، در یک نمای واحد و رایگان برای جست‌وجو قرار می‌دهد. اثر مستقیم آن کاهش نابرابری اطلاعاتی میان خریدار و فروشنده تبلیغات است: کسب‌وکار کوچکی که بودجه محدودی دارد می‌تواند پیش از تماس با هر واسطه، دامنه قیمت و گزینه‌های شهر خود را ببیند.

در طراحی، سه ملاحظه اخلاقی صریح رعایت شده است. نخست، **حفاظت از داده شخصی**: شماره تماس مالکان در صفحه عمومی قرار نمی‌گیرد و تنها پس از احراز هویت و ثبت درخواست در دسترس است، تا فهرست تماس‌ها به‌صورت انبوه برداشت نشود. دوم، **صداقت در داده**: رکوردهایی که از منابع عمومی گردآوری شده‌اند به‌عنوان داده گردآوری‌شده نشانه‌گذاری شده‌اند و به‌جای آنکه ادعای دقت تأییدشده داشته باشند، وضعیت کیفیت‌شان در پنل مدیریت قابل پیگیری است. سوم، **پرهیز از ادعای نادرست**: سامانه واسطه مالی نیست و مالک رسانه‌ها نیست، بنابراین عمداً جریان «رزرو آنلاین» ندارد؛ دلیل این تصمیم در بخش ۴-۳ آمده است.

محدودیت‌های صادقانه کار – از جمله اینکه داده اولیه یک گردآوری یک‌باره نقطه‌ای است و نه جریانی که خودش به‌روز بماند – در فصل ۷ فهرست شده‌اند تا خواننده تصویر واقع‌بینانه‌ای از دامنه اعتبار نتایج داشته باشد.

فهرست مطالب

۱	فصل ۱ مقدمه
۱	۱-۱ بیان مسئله
۱	۲-۱ اهمیت و انگیزه
۱	۳-۱ مزیت رقابتی و نوآوری
۲	۴-۱ اهداف پژوهش
۳	۵-۱ دامنه و ساختار
۴	فصل ۲ پیشینه و مرور فناوری‌ها
۴	۱-۲ بازار رسانه‌های محیطی و منابع داده موجود
۴	۲-۲ سامانه‌های مشابه
	۳-۲ انتخاب پشته فناوری
۷	فصل ۳ تحلیل نیازمندی‌ها
۷	۱-۳ ذی‌نفعان و نقش‌ها
۷	۲-۳ نیازمندی‌های وظیفه‌ای
۸	۳-۳ نیازمندی‌های غیروظیفه‌ای
۸	۴-۳ موارد کاربرد اصلی
۸	۵-۳ آنچه عمداً ساخته نشد
۱۰	فصل ۴ طراحی سامانه
۱۰	۱-۴ معماری کلی و دو مسیر داده
۱۱	۲-۴ موتور پایگاه داده و مدل داده
۱۳	۳-۴ طراحی واسط برنامه‌نویسی
۱۴	۴-۴ طراحی رابط کاربری
	۵-۴ سازمان‌دهی کد و نقشه وابستگی‌ها
۱۸	۶-۴ گراف وابستگی پروژه
۲۳	فصل ۵ پیاده‌سازی
۲۳	۱-۵ پایپلاین ثابت هر درخواست
۲۴	۲-۵ احراز هویت و کنترل دسترسی
۲۶	۳-۵ ثبت آگهی و بارگذاری امن فایل
	۴-۵ محافظت از داده تماس مالکان
۲۸	۵-۵ بازیابی گذرواژه با رمز یک‌بارمصرف
۲۹	۶-۵ همروندی و صحت زیر بار همزمان
۳۱	۷-۵ مدل تخمین بازدید روزانه
	۸-۵ مدیریت خطا و ثبت رخداد
۳۴	فصل ۶ ارزیابی و آزمون
۳۴	۱-۶ روش و نتایج آزمون خودکار

۳۵	۲-۶ یک کلاس خطا و درسی که از آن گرفته شد
۳۶	۳-۶ بازبینی بدبینانه معماری
۳۷	۴-۶ خودارزیابی
۳۹	فصل ۷ نتیجه‌گیری و کار آینده
۳۹	۱-۷ دستاوردها
۳۹	۲-۷ محدودیت‌ها
۳۹	۳-۷ کار آینده
۴۰	۴-۷ جمع‌بندی
۳۹	فهرست مراجع
۴۲	فصل ۸ راهنمای اجرای پروژه
۴۲	۱-۸ پیش‌نیازها و راه‌اندازی
۴۳	فصل ۹ نماهایی از سامانه

فهرست شکل‌ها

- شکل ۴-۱ معماری کلی سامانه. صرف‌نظر از اینکه درخواست از کدام مسیر بیاید، در نهایت از یک Data Access Layer (لایه واحد دسترسی به داده) رد می‌شود؛ Proxy پیش از هر چیز روی مسیر است، و هر درخواست /api/ پیش از رسیدن به لایه داده از یک خط لوله واحد می‌گذرد. ۱۰
- شکل ۴-۲ نمودار موجودیت-رابطه (ERD): هر جعبه یک جدول است و هر خط یک پیوند کلید خارجی میان دو جدول. سه جدول - OTP_CODES، IDEMPOTENCY_KEYS و SOURCE_TOMBSTONES - به هیچ جدول دیگری وصل نیستند، چون به کسب‌وکار ربطی ندارند و فقط زیرساخت صحت‌اند. رد ممیزی، برعکس، انجام‌دهنده را با کلید خارجی به جدول خودش (کارمند یا مشتری) وصل می‌کند. ۱۱
- شکل ۴-۳ صفحه جست‌وجو و پالایش. پالایه‌های شهر، نوع، قیمت و ابعاد بالای شبکه نتایج؛ هر کارت قیمت ماهانه و برآورد بازدید روزانه را نشان می‌دهد. ۱۲
- شکل ۴-۴ صفحه جزئیات رسانه: گالری تصویر، مشخصات فنی، موقعیت روی نقشه، و پنل «آنالیز ترافیک» که برآورد بازدید روزانه و مبنای محاسبه‌اش را می‌آورد. شماره تماس در این صفحه نیست؛ جایش دکمه «برای دیدن اطلاعات تماس وارد شوید» است. ۱۵
- شکل ۴-۵ ساختار پوشه‌های پروژه. نکته کلیدی این چیدمان این است که /lib/db تنها راه رسیدن به پایگاه داده است - و این فقط یک قرار نیست؛ قاعده ESLint واردکردن اتصال پایگاه داده را بیرون از این پوشه، و هر واردکردن دارای ورودی/خروجی را در /lib/domain، به‌عنوان خطا رد می‌کند. ۱۶
- شکل ۴-۶ سفر یک درخواست، از کلیک کاربر تا پایگاه داده و برگشت. فایل مسیر هیچ منطقی ندارد؛ خط لوله بررسی‌ها را به ترتیب ثابت اجرا می‌کند و فقط ورودی سالم را به لایه داده می‌دهد. صفحه‌ها همین لایه داده را بی‌واسطه صدا می‌زنند (مسیر یکم) و از خط لوله نمی‌گذرند، چون درخواست HTTP نیستند. ۱۷
- شکل ۴-۷ وابستگی میان لایه‌های کد؛ عدد هر پیکان تعداد import است. هر ۳۵ مسیر به خط لوله وصل‌اند (۵۲ پیکان) و بیشتر کار احراز هویت را از آن می‌گیرند. نکته مهم‌تر آن است که هیچ پیکانی از مسیرها یا صفحه‌ها به خود پایگاه داده نمی‌رود؛ هر دو فقط به Data Access Layer وصل‌اند - مسیرها از راه خط لوله، و صفحه‌ها بی‌واسطه (مسیر یکم بخش ۴-۱). لایه دامنه هم هیچ پیکان خروجی‌ای ندارد؛ به چیزی جز خودش وابسته نیست. ۱۸
- شکل ۴-۸ گراف وابستگی در سطح فایل - نمایی خلاصه: از هر لایه چند فایل بزرگ‌تر به‌اضافه فایل‌های نام‌گذاری‌شده (۴۲ فایل از ۱۷۳ فایل متصل، ۷۴ پیوند). آنچه باید دید، تراکم دایره‌ها نزدیک مرکز سه نوار پایینی - احراز هویت، ابزارهای مشترک و Data Access Layer - است؛ هرچه فایلی به مرکز نوارش نزدیک‌تر باشد، فایل‌های بیشتری واردش می‌کنند. همین تصویر است که جدول بعدی به‌صورت عدد نشان می‌دهد. ۱۹
- شکل ۴-۹ نمای کلی پنل مدیریت. اعداد این صفحه مستقیم از پایگاه داده خوانده می‌شوند و همان‌هایی‌اند که در سراسر این نوشته نقل شده‌اند: ۳,۵۳۶ رسانه، ۳,۰۲۰ دارای مختصات (۸۵٪)، و سهم هر منبع. ۲۰
- شکل ۴-۱۰ همان رابط روی نمایشگر تلفن همراه. چیدمان، پالایه‌ها و کارت‌ها برای عرض کم بازآرایی می‌شوند، نه اینکه کوچک شوند. ۲۲

- شکل ۵-۱** خط لوله ثابت هر درخواست. ترتیب دروازه‌ها عوض نمی‌شود؛ چرایی‌اش – که امنیتی است، نه سلیقه‌ای – درست پس از همین شکل آمده است. لایه داده هم خودش پاسخ HTTP نمی‌سازد: قاعده شکسته‌شده را به صورت DomainError با پیام فارسی اعلام می‌کند و خط لوله آن را به کد وضعیت ترجمه می‌کند.
- قطعه کد ۵-۱** خط لوله در عمل، از مسیر تصمیم کارشناس. مسیر فقط اعلام می‌کند چه می‌خواهد؛ زمان اجرای هر بررسی را خط لوله تعیین می‌کند، پس هیچ مسیری نمی‌تواند ترتیب را جابه‌جا کند.
- شکل ۵-۲** بستن نشن زما: در حالت «کاربر وجود ندارد» هم یک هش الکی روی یک مقدار ثابت گرفته می‌شود، تا مدت پاسخ نگوید کدام شماره در سامانه ثبت است.
- شکل ۵-۲** جریان ورود. پیام خطا در هر دو حالت «شماره ناموجود» و «گذرواژه اشتباه» یکسان است تا شمارش کاربران ممکن نشود.
- قطعه کد ۵-۳** تشخیص نوع تصویر از بایت‌های آغازین خود فایل. نوع اعلام‌شده مرورگر اصلاً مبنا قرار نمی‌گیرد.
- شکل ۵-۳** ثبت آگهی. کلید یکتایی تضمین می‌کند ارسال دوباره فرم، آگهی دوم نسازد.
- شکل ۵-۴** فرم ثبت آگهی از دید صاحب رسانه. هر مرحله پیش از رفتن به بعدی اعتبارسنجی می‌شود و دکمه ارسال هنگام پرواز درخواست غیرفعال می‌ماند.
- شکل ۵-۵** دریافت شماره تماس. سه لایه (نبود شماره در صفحه، الزام ورود، و سقف درخواست) برداشت انبوه را پرهزینه می‌کنند.
- شکل ۵-۶** بازیابی گذرواژه. لایه پیامک بدون کلید سرویس در حالت خواب می‌ماند؛ کد کامل است ولی پیامی ارسال نمی‌شود.
- قطعه کد ۵-۴** تصمیم تک‌شلیک. شرط و نوشتن یک عملیات اتمی‌اند، پس تلاش دوم برای همان گذار پاسخ «تعارض» می‌گیرد – حتی اگر دقیقاً هم‌زمان برسد – و تأیید فقط روی همان نسخه‌ای می‌نشیند که کارشناس دیده است. همین الگو برای ارسال دوباره آگهی و برای مصرف کد یک‌بارمصرف نیز به کار رفته است. خود قاعده گذار (decisionOutcome) یک تابع خالص در lib/domain است و آزمون واحد دارد.
- شکل ۵-۷** ماشین حالت یک آگهی. نام انگلیسی هر وضعیت همان است که در پایگاه داده ذخیره می‌شود. چون هر پیکان تنها از یک وضعیت آغاز می‌شود، دو کارشناس نمی‌توانند یک آگهی را دو بار تأیید کنند: تلاش دوم پاسخ «تعارض» (کد ۴۰۹) می‌گیرد.
- شکل ۵-۸** همان ماشین حالت در عمل: نشان‌های «در انتظار تأیید»، «رد شده» و «در انتظار پرداخت» کنار سه تصمیم مجاز کارشناس. توضیح برای فرستنده در حالت «نیاز به اصلاح» و «رد» الزامی است.
- شکل ۹-۱** سمت کاربر: صفحه نخست با آمار کلی و رسانه‌های شاخص، و پیشخوانی که وضعیت هر آگهی ثبت‌شده را در چرخه تأیید نشان می‌دهد.
- شکل ۹-۲** صفحه تحلیل، توزیع شهر، نوع و قیمت را از داده واقعی رسم می‌کند. مرجع کامل هر ۴۴ نقطه پایانی هم از دل خود سامانه ارائه می‌شود و به سند بیرونی نیازی نیست.
- شکل ۹-۳** پنل مدیریت: ویرایش رسانه‌ها، و صفحه کیفیت که رکوردهای ناقص (بدون تصویر یا بدون مختصات) را برای پیگیری جدا می‌کند.
- شکل ۹-۴** چهار سطح دسترسی مدیران، و رد ممیزی که هر تغییر مدیریتی را با شناسه مدیر، نشانی شبکه و جزئیات تغییر نگه می‌دارد.

فهرست جدول‌ها

۲	جدول ۱-۱ مقایسه ساختاری با وب‌گاه‌های تک‌فروشنده موجود
۴	جدول ۲-۱ سهم منابع در پایگاه داده نهایی، پس از حذف رکوردهای تکراری
۵	جدول ۲-۲ انتخاب‌های فناوری، بدیل بررسی‌شده و دلیل انتخاب
۷	جدول ۳-۱ نقش‌های سامانه، هدف هر نقش و مرز دسترسی آن
۷	جدول ۳-۲ نیازمندی‌های وظیفه‌ای سامانه
۸	جدول ۳-۳ نیازمندی‌های غیروظیفه‌ای و معیار پذیرش هر یک
۱۴	جدول ۴-۱ توزیع نقاط پایانی بر حسب فعل HTTP
۱۶	جدول ۴-۲ لایه‌های کد، تعداد فایل و نقش هر لایه
۲۱	جدول ۴-۳ پرجام‌ترین فایل‌های پروژه – چند فایل به هر یک وابسته‌اند
۳۲	جدول ۵-۱ کلاس‌های معبر و ضرایب هر یک. ستون «سهم دیدن» نسبت جمعیتی است که امکان واقعی خواندن پیام را دارد.
۳۴	جدول ۶-۱ سرفصل‌های پوشش آزمون خودکار
۳۵	جدول ۶-۲ شش خطای هم‌ریشه و اصلاح هر یک
۳۶	جدول ۶-۳ یافته‌های بازبینی بدبینانه و آنچه جایشان نشست
۳۷	جدول ۶-۴ خودارزیابی محورهای پروژه
۴۰	جدول ۷-۱ کارهای آینده به ترتیب اولویت

فهرست اختصارها و واژه‌نامه

هر اصطلاح فنی همان‌جا که نخستین‌بار در متن می‌آید کوتاه توضیح داده شده است؛ این جدول فقط برای مرور سریع است.

API مجموعه‌ای از آدرس‌ها که یک برنامه در اختیار برنامه‌های دیگر می‌گذارد تا از آن داده بگیرند یا به آن داده

بدهند. در این پروژه، همان آدرس‌هایی که با `/api/` آغاز می‌شوند.

CSRF حمله‌ای که در آن یک سایت مهاجم، از طرف کاربری که همین حالا در سایت ما وارد است، پنهانی برایش

درخواست می‌فرستد؛ مرورگر چون کوکی کاربر را دارد آن را معتبر می‌شمارد.

ERD نقشه‌ای که جدول‌های پایگاه داده و پیوند میان‌شان را نشان می‌دهد؛ کدام جدول به کدام وصل است و

رابطه یک‌به‌چند است یا چندبه‌چند.

JWT یک رشته متنی امضا شده که پس از ورود به کاربر داده می‌شود و در هر درخواست بعدی همراهش

می‌آید؛ سرور با بررسی امضا می‌فهمد کاربر واقعاً وارد شده، بی‌آنکه لازم باشد نشست را جایی نگه دارد.

ORM لایه‌ای که به جای نوشتن دستی دستورهای SQL، ردیف‌های جدول را همچون شی‌های معمولی برنامه

در اختیار می‌گذارد. اینجا نقش آن را Prisma دارد.

OTP رمزی که فقط یک بار و تا مدت کوتاهی معتبر است. اینجا برای بازیابی گذرواژه با پیامک فرستاده

می‌شود.

RBAC روشی برای تعیین دسترسی که در آن به جای اجازه‌دادن به تک‌تک کاربران، به هر «نقش» (مهمان، کاربر،

کارشناس، مدیر ارشد) مجموعه‌ای از اجازه‌ها داده می‌شود و هر کاربر یکی از این نقش‌ها را دارد.

RSC React Server Component – قطعه‌ای از رابط کاربری که فقط روی سرور اجرا می‌شود و خروجی‌اش

HTML آماده است؛ کدش هرگز به مرورگر فرستاده نمی‌شود.

RTL چیدمان راست‌به‌چپ – جهتی که فارسی با آن نوشته می‌شود و کل رابط این پروژه بر همان اساس

طراحی شده است.

SSR ساختن صفحه آماده روی سرور و فرستادن آن به مرورگر، به جای فرستادن یک صفحه خالی که بعداً در

مرورگر با JavaScript پر شود.

WAL حالتی از SQLite که نوشتن‌ها را در یک فایل جداگانه نگه می‌دارد؛ در نتیجه خواننده‌ها لازم نیست پشت

یک نویسنده منتظر بمانند و خواندن و نوشتن هم‌زمان ممکن می‌شود.

Audit log رد ممیزی – دفتری که هر کار حساس مدیران (تأیید، رد، حذف، تغییر نقش) را با زمان، شناسه مدیر و

نشانی شبکه در آن ثبت می‌کنند و بعداً پاک یا ویرایش نمی‌شود.

Idempotency این خاصیت که اگر کاربر یک درخواست را دو بار بفرستد، کار فقط یک بار انجام شود – مثلاً یک آگهی

ساخته شود نه دو تا. اینجا با یک «کلید یکتا» همراه هر ثبت آگهی تأمین شده است.

Middleware / کدی که پیش از رسیدن هر درخواست به مقصدش اجرا می‌شود و می‌تواند آن را رد کند یا اجازه عبور

Proxy بدهد. در Next.js 16 نامش از Middleware به Proxy تغییر کرده است.

Rate limiting سقفی روی تعداد درخواستی که یک نفر در بازه زمانی معین می‌تواند بفرستد؛ ابزار اصلی برای مهار حمله

حدس گذرواژه و برداشت انبوه داده.

State machine ماشین حالت – توصیف یک چیز به صورت مجموعه‌ای از وضعیت‌ها و فهرست گذارهای مجاز میان

آن‌ها. وضعیت یک آگهی (در انتظار، منتشرشده، نیازمند اصلاح...) با همین الگو مدل شده است.

Transaction تراکنش – گروهی از تغییرها روی پایگاه داده که یا همه با هم انجام می‌شوند یا هیچ‌کدام؛ حالت

نیمه‌کاره ممکن نیست.

مقدمه

وضعیت پروژه در زمان نگارش

این سامانه یک نمونه کارآمد و در حال اجرا است، نه یک محصول مستقرشده. تا این تاریخ روی هیچ دامنه عمومی مستقر نشده و روی رایانه توسعه اجرا می‌شود. بخش‌هایی از آن عمداً ساخته نشده‌اند (رزرو آنلاین، درگاه پرداخت، به‌روزرسانی خودکار داده) و دلیل هر یک در متن آمده است. آنچه ساخته شده کار می‌کند و آزموده شده؛ آنچه نشده، فهرست و توجیه شده است. مسیر تکمیل آن نیز روشن و بدون نیاز به بازطراحی است (فصل ۷).

۱-۱ بیان مسئله

تبلیغات محیطی (بیلبورد، تابلوی پل عابر، استرابورد ایستگاه و نمایشگر شهری) یکی از پرمخاطب‌ترین رسانه‌های تبلیغاتی است، اما برخلاف تبلیغات آنلاین که از یک پیشخوان واحد خریداری می‌شود، خرید آن در ایران هنوز فرایندی تلفنی و واسطه‌محور است.

ریشه مسئله در ساختار مالکیت است: هر سازه در اختیار یک شهرداری، پیمانکار یا شرکت تبلیغاتی است و فهرست موجودی، قیمت و وضعیت آن نزد همان مالک می‌ماند. خریداری که می‌خواهد بداند در شهر خود چه گزینه‌هایی با چه ابعاد و قیمتی وجود دارد، ناچار است با چند واسطه جداگانه تماس بگیرد و پاسخ‌هایی را که در قالب‌های متفاوت داده می‌شود دستی مقایسه کند. پیامد عملی این پراکندگی سه چیز است: زمان تلف‌شده، نبود امکان مقایسه منصفانه، و نابرابری اطلاعاتی به سود فروشندگان.

صورت فشرده مسئله

عرضه رسانه محیطی در ایران وجود دارد و کم هم نیست؛ آنچه وجود ندارد یک نمای واحد و قابل جست‌وجو از این عرضه است. مسئله این پایان‌نامه ساختن آن نما است.

۲-۱ اهمیت و انگیزه

در بازارهایی که عرضه پراکنده است، اثر یک فهرست متمرکز صرفاً «راحتی» نیست؛ ترکیب بازار را تغییر می‌دهد. کسب‌وکار کوچکی که بودجه محدودی دارد امروز حتی نمی‌داند در شهر خودش گزینه‌ای در توان مالی او هست یا نه، چون هزینه جست‌وجو برایش بالاتر از ارزش احتمالی نتیجه است؛ یک فهرست قابل جست‌وجو این هزینه را تقریباً صفر می‌کند. از منظر مهندسی نرم‌افزار نیز مسئله تمرین کاملی است: داده واقعی و کثیف، دو گروه کاربر با نیازهای متضاد، محتوای تولیدشده توسط کاربر که باید پیش از انتشار بررسی شود، داده شخصی‌ای که باید محافظت شود، و رابطی راست‌به‌چپ که بیشتر ابزارهای رایج برای آن بهینه نشده‌اند.

۳-۱ مزیت رقابتی و نوآوری

پیش از آنکه اهداف فهرست شود، باید روشن باشد این سامانه دقیقاً چه چیزی به وضعیت موجود اضافه می‌کند. وب‌گاه‌های امروزی رسانه محیطی در ایران عمدتاً وب‌گاه یک شرکت‌اند، نه یک بازار: هر یک موجودی

خودش را نشان می‌دهد و (همان‌طور که در گردآوری داده مشاهده شد) برای هر سازه یک عکس، یک اندازه، یک نام خیابان و گاهی یک قیمت منتشر می‌کند و نه بیشتر.

جدول ۱-۱ – مقایسه ساختاری با وبگاه‌های تک‌فروشنده موجود

ویژگی	وبگاه‌های موجود	این سامانه
دامنه موجودی	فقط سازه‌های همان شرکت	تجمیع سه منبع در یک فهرست واحد: ۳,۵۳۶ رسانه در ۱۰۱ شهر
جست‌وجوی یکپارچه	کاربر باید چند وب‌گاه را جداگانه بگردد	یک پالایه واحد روی شهر، نوع، دامنه قیمت، ابعاد و وجود تصویر
نقشه	معمولاً فقط نام خیابان به صورت متن	نمایش هم‌زمان فهرست و نقشه؛ ۳,۰۲۰ رسانه دارای مختصات جغرافیایی (۸۵/۴٪)
مقایسه کنار هم	وجود ندارد	انتخاب چند رسانه و مقایسه ستونی مشخصات، قیمت و بازدید
تخمین بازدید روزانه	وجود ندارد	مدل شفاف مبتنی بر کلاس معبر – بخش ۷-۵
شفافیت قیمت	پراکنده؛ گاهی «تماس بگیرید»	قیمت در خود فهرست، قابل مرتب‌سازی و پالایش
ثبت آگهی خودخدمت	معمولاً تماس تلفنی	فرم چندمرحله‌ای با بارگذاری تصویر و چرخه تأیید کارشناس
حفاظت از داده تماس	شماره در صفحه عمومی	پشت احراز هویت، با ثبت درخواست و سقف نرخ
فارسی و راست‌به‌چپ	غالباً قالب ترجمه‌شده با ایرادهای چیدمانی	راست‌به‌چپ بومی در تمام چیدمان، جدول‌ها، نمادها و انیمیشن‌ها

نقطه تمایز اصلی – تخمین بازدید روزانه

هیچ‌یک از منابع بررسی‌شده عددی برای «چند نفر در روز این سازه را می‌بینند» منتشر نمی‌کند، در حالی که این دقیقاً عددی است که خریدار تبلیغات برای سنجش ارزش خرید به آن نیاز دارد. در این پروژه مدلی ساخته شده که این عدد را از داده‌ای که موجود است (کلاس معبر، اندازه سازه، نوع رسانه و شهر) برآورد می‌کند.

مدل از همان سه‌گامی پیروی می‌کند که صنعت جهانی تبلیغات محیطی به کار می‌برد (سامانه‌های Geopath در آمریکا و Route در بریتانیا): شمارش عبور روزانه، تبدیل آن به تعداد نفر، و در پایان ضرب در یک ضریب کوچک‌تر از یک که می‌گوید چه سهمی از این جمعیت واقعاً امکان دیدن همین سازه مشخص را دارد. خروجی آن **قطعی و بازتولیدپذیر** است (یک رسانه در دو اجرای متفاوت دقیقاً یک عدد می‌گیرد) و منابع هر ضریب در کد ثبت شده است. شرح کامل روش در بخش ۷-۵ آمده است.

۴-۱ اهداف پژوهش

هدف کلی، طراحی و پیاده‌سازی یک سامانه تحت وب کارا و امن برای جست‌وجو و ثبت آگهی رسانه‌های تبلیغاتی محیطی است. اهداف جزئی:

۱. گردآوری و یکپارچه‌سازی داده رسانه‌ها از منابع عمومی موجود در یک پایگاه داده واحد؛

2. جست‌وجو، پالایش و مقایسه بر پایه شهر، نوع، ابعاد، قیمت و موقعیت جغرافیایی؛
3. چرخه خودخدمت ثبت آگهی، همراه با بررسی و تأیید کارشناس پیش از انتشار؛
4. محافظت از داده تماس مالکان در برابر برداشت انبوه، بدون دشوار کردن دسترسی کاربر واقعی؛
5. کیفیت مهندسی قابل دفاع در برابر خطا، سوءاستفاده و بار هم‌زمان؛
6. سنجش صحت سامانه با آزمون خودکار روی نتایج واقعی، نه ارزیابی حدسی.

۵-۱ دامنه و ساختار

مرز این پروژه آگاهانه تعیین شده است. سامانه یک **فهرست و بازارگاه اطلاعاتی** است: کاربر را تا لحظه تماس با مالک می‌رساند و توافق از آنجا بیرون از سامانه انجام می‌شود. سامانه مالک هیچ رسانه‌ای نیست و تقویم اشغال هیچ سازه‌ای را در اختیار ندارد، بنابراین جریان «رزرو آنلاین» عمداً ساخته نشده است (بخش ۵-۳). همچنین برای بار یک نمونه اجرایی و ترافیک خواندن محور طراحی شده، نه برای مقیاس چندسروری (بخش ۲-۴ و فصل ۷).

فصل ۲ بازار هدف، سامانه‌های مشابه و دلیل انتخاب هر فناوری را بررسی می‌کند؛ **فصل ۳** ذی‌نفعان، نیازمندی‌ها و موارد کاربرد را استخراج می‌کند؛ **فصل ۴** طراحی را در چهار سطح معماری، مدل داده، واسط برنامه‌نویسی و رابط کاربری ارائه می‌دهد؛ **فصل ۵** به پیاده‌سازی و تصمیم‌های مهندسی می‌پردازد؛ **فصل ۶** روش و نتایج ارزیابی را گزارش می‌کند و **فصل ۷** جمع‌بندی می‌کند.

پیشینه و مرور فناوری‌ها

۱-۲ بازار رسانه محیطی و منابع داده موجود

در ایران چند وبگاه تخصصی وجود دارد که موجودی رسانه‌های محیطی را منتشر می‌کنند. این وبگاه‌ها معمولاً متعلق به یک شرکت تبلیغاتی‌اند و طبیعتاً فهرست خودشان را نشان می‌دهند، نه فهرست بازار را. افزون بر آن، آگهی‌های پراکنده‌ای هم در بسترهای عمومی نیازمندی درج می‌شود که ساختار یکسانی ندارند.

برای این پروژه داده اولیه از سه منبع عمومی گردآوری شد. جدول زیر سهم هر منبع را در پایگاه داده نهایی نشان می‌دهد.

جدول ۱-۲ - سهم منابع در پایگاه داده نهایی، پس از حذف رکوردهای تکراری

منبع	تعداد رکورد	سهم
billboardiha	۲,۷۸۱	۷۸٪
aradholding	۶۱۵	۱۷,۴٪
irbillboard	۱۱۵	۳,۳٪
ثبت‌شده توسط کاربران سامانه	۸	۰,۲٪
بدون منبع مشخص	۱۷	۰,۵٪
مجموع	۳,۵۳۶	۱۰۰٪

نکته مهم آنکه این رکوردها یک گردآوری یک‌باره نقطه‌ای است، نه جریانی که خودکار به‌روز شود. هر رکورد با برچسب منبع خود نگهداری می‌شود تا هم قابل ردیابی باشد و هم بعداً بتوان همان منبع را به‌روزرسانی کرد. رکوردهایی که در بیش از یک منبع تکرار شده بودند با مقایسه نام، شهر و موقعیت جغرافیایی شناسایی و ادغام شدند.

۲-۲ سامانه‌های مشابه

بازارگاه‌های تبلیغات محیطی معمولاً یکی از دو الگو را دنبال می‌کنند. الگوی نخست، بازارگاه معاملاتی است که موجودی و تقویم اشغال سازه‌ها را در اختیار دارد و خرید را تا پرداخت آنلاین پیش می‌برد. الگوی دوم، فهرست اطلاعاتی است که عرضه را قابل جست‌وجو می‌کند و طرفین را به هم می‌رساند بی‌آنکه خودش طرف معامله باشد.

الگوی نخست تنها زمانی کار می‌کند که سامانه به تقویم واقعی اشغال دسترسی داشته باشد، یعنی یا مالک باشد یا با مالکان یکپارچگی داده‌ای داشته باشد. در نبود این دسترسی، «رزرو آنلاین» چیزی جز یک وعده بی‌پشتوانه نیست. به همین دلیل الگوی دوم انتخاب شده است؛ این یک محدودیت فنی نیست، یک تصمیم طراحی است (بخش ۵-۳).

۳-۲ انتخاب پشتۀ فناوری

هر انتخاب فناوری در این پروژه در برابر دستکم یک بدیل جدی سنجیده شده است. جدول زیر خلاصۀ این سنجش است.

جدول ۲-۲ – انتخاب‌های فناوری، بدیل بررسی‌شده و دلیل انتخاب

لایه	انتخاب	بدیل	دلیل
چارچوب	Next.js 16 (App Router)	Django + DRF با یک رابط جدا	رابط کاربری، مسیرهای برنامه‌نویسی و رندر سمت سرور در یک پایگاه کد؛ یک بار استقرار به‌جای دو سرویس. برای یک توسعه‌دهندۀ تنها با مهلت محدود، حذف مرز شبکه میان دو سرویس مهم‌ترین صرفه‌جویی است.
زبان	TypeScript در حالت strict	JavaScript	خطاهای نوع در زمان ساخت گرفته می‌شوند، نه در جلسۀ دفاع. هزینه‌اش کندتر نوشتن است و در پروژه‌ای با یک نویسنده، ارزشش را دارد.
پایگاه داده	SQLite	PostgreSQL	بار خواندن‌محور، یک نمونۀ اجرایی، بدون نیاز به سرویس جداگانه. مهاجرت بعدی هزینه‌بازنویسی ندارد – بخش ۲-۴.
لایۀ داده	Prisma 7	SQL دستی	طرح‌واره‌نوع‌دار، مهاجرت نسخه‌بندی‌شده، و اینکه چون پرس‌وجو دستی به‌صورت رشته ساخته نمی‌شود، جای تزریق SQL از بیخ بسته است.
اعتبارسنجی	Zod	بررسی دستی با if	یک طرح‌واره هم نوع زمان ساخت را می‌دهد و هم بررسی زمان اجرا را؛ یک تعریف واحد به‌جای دو.
احراز هویت	توکن امضاشده در کوکی HttpOnly	سرویس احراز هویت میزبانی‌شده	پرهیز از وابستگی به سرویس پولی یا تحریم‌شده. کد آن حدود دویست خط است و کاملاً قابل بازرسی.
نقشه	نقشه گوگل، به‌صورت iframe جاسازی‌شده	Leaflet + کاشی متن‌باز	Leaflet روی سرور این پروژه (لپ‌تاپ بدون فن) سنگین بود. جاسازی نقشۀ گوگل بدون کلید و با بار ناچیز همان کار را انجام می‌دهد؛ بهایش وابستگی به اینترنت است و هنگام قطعی کار نمی‌کند، ولی پس از بررسی گزینه‌ها بهترین موازنه بود. مختصات هر رسانه پیش‌تر با سرویس ژئوکدینگ نشان به دست آمده و در پایگاه داده ذخیره شده است.

قید حاکم بر همه انتخاب‌ها

در سراسر پروژه هیچ سرویس پولی یا مسدودشده برای ایران به کار نرفته است. هر جا بهترین‌روش رایج به چنین سرویسی نیاز داشته، یا معادل درون‌کدی آن ساخته شده، یا از آن صرف‌نظر و در مستندات ثبت شده که چه چیزی از

دست رفته است. این قید کیفیت مهندسی را پایین نیاورده، ولی شکل بعضی راه‌حل‌ها را تعیین کرده است.

تحلیل نیازمندی‌ها

۱-۳ ذی‌نفعان و نقش‌ها

سامانه چهار نقش دارد که دسترسی هر یک جداگانه تعریف شده است:

جدول ۱-۳ – نقش‌های سامانه، هدف هر نقش و مرز دسترسی آن

نقش	هدف	مرز دسترسی
مهمان	ارزیابی اینکه آیا سامانه به دردش می‌خورد	جست‌وجو، پالایش، دیدن جزئیات و نقشه. شماره تماس مالک را نمی‌بیند.
خریدار تبلیغات (کاربر واردشده)	یافتن رسانه مناسب و تماس با مالک	همه دسترسی‌های مهمان، به‌علاوه دریافت شماره تماس، ثبت نظر و مقایسه.
صاحب رسانه	معرفی رسانه خود به خریداران	ثبت آگهی با تصویر، پیگیری وضعیت آن در پیشخوان، و اصلاح آگهی در صورت درخواست کارشناس.
کارشناس / مدیر	حفظ کیفیت و صحت فهرست	چهار سطح پلکانی: viewer فقط می‌بیند، editor ویرایش می‌کند، admin تأیید و رد می‌کند، super_admin مدیران را مدیریت می‌کند.

۲-۳ نیازمندی‌های وظیفه‌ای

جدول ۲-۳ – نیازمندی‌های وظیفه‌ای سامانه

شناسه	نیازمندی	شرح
FR-1	جست‌وجو و پالایش	پالایش بر پایه شهر، نوع رسانه، دامنه قیمت، ابعاد و وجود تصویر؛ به‌همراه مرتب‌سازی و صفحه‌بندی.
FR-2	نمایش روی نقشه	نمایش هم‌زمان فهرست و نقشه، با موقعیت جغرافیایی هر رسانه.
FR-3	صفحه جزئیات	گالری تصویر، مشخصات فنی، برآورد بازدید روزانه، موقعیت و نظرات کاربران.
FR-4	مقایسه	انتخاب چند رسانه و مقایسه کنار هم مشخصات آن‌ها.
FR-5	ثبت‌نام و ورود	ثبت‌نام با شماره موبایل و گذرواژه؛ بازیابی گذرواژه با رمز یک‌بارمصرف پیامکی.
FR-6	دریافت شماره تماس	نمایش شماره مالک تنها به کاربر واردشده، همراه با ثبت درخواست.
FR-7	ثبت آگهی	فرم چندمرحله‌ای با بارگذاری تصویر و انتخاب یکی از دو پلن: رایگان، یا ویژه (با پرداخت و نشان برجسته).

شناسه	نیازمندی	شرح
FR-8	چرخه تأیید	ماشین حالت آگهی از ثبت تا انتشار، با امکان درخواست اصلاح.
FR-9	نظر و امتیاز	ثبت یک نظر برای هر کاربر روی هر رسانه، با امکان پاسخ مدیر.
FR-10	پنل مدیریت	مدیریت رسانه‌ها، صف تأیید، وضعیت کیفیت داده، کاربران و رد ممیزی.

۳-۳ نیازمندی‌های غیروظيفه‌ای

جدول ۳-۳ – نیازمندی‌های غیروظيفه‌ای و معیار پذیرش هر یک

شناسه	نیازمندی	معیار پذیرش
NFR-1	امنیت ورودی	هیچ ورودی کاربر بدون اعتبارسنجی طرح‌واره‌ای به لایه داده نرسد.
NFR-2	کنترل دسترسی	مرز دسترسی روی خود مسیر برنامه‌نویسی اعمال شود، نه فقط در رابط کاربری.
NFR-3	مقاومت در برابر سوءاستفاده	سقف نرخ درخواست روی ورود، ثبت‌نام، بازیابی گذرواژه و مسیرهای پرهزینه.
NFR-4	صحت زیر بار هم‌زمان	ارسال دوباره یک درخواست نباید اثرش را دو بار اعمال کند.
NFR-5	مدیریت خطا	کاربر هرگز صفحه سفید یا رد پشت‌خام نبیند؛ هر خطای پیش‌بینی نشده شناسه پیگیری داشته باشد.
NFR-6	زبان، جهت و موبایل	تمام متن دیدنی کاربر فارسی و چیدمان راست‌به‌چپ باشد و همه صفحه‌ها روی نمایشگر تلفن همراه قابل استفاده باشند.
NFR-7	کارایی	سامانه روی یک رایانه شخصی معمولی و بدون شتاب‌دهنده بیرونی روان اجرا شود.
NFR-8	ردیابی پذیری	هر تغییر مدیریتی در یک رد ممیزی ماندگار ثبت شود.

۴-۳ موارد کاربرد اصلی

پنج مورد کاربرد، ستون فقرات سامانه‌اند. هر یک در فصل ۵ با نمودار توالی کامل ارائه شده است:

1. جست‌وجوی رسانه – بازدیدکننده شهر و نوع را انتخاب می‌کند و به فهرست پالایش‌شده می‌رسد.
2. ورود به سامانه – کاربر با شماره و گذرواژه وارد می‌شود و نشست امضا شده دریافت می‌کند (شکل ۵-۲).
3. دریافت شماره تماس – کاربر وارد شده شماره مالک را می‌گیرد و درخواستش ثبت می‌شود (شکل ۵-۴).
4. ثبت آگهی توسط صاحب رسانه – فرم چندمرحله‌ای، بارگذاری تصویر، و ورود به صف بررسی (شکل ۵-۳).
5. بازیابی گذرواژه – کاربر با رمز یک‌بارمصرف پیامکی گذرواژه‌اش را بازنشانی می‌کند (شکل ۵-۵).

۵-۳ آنچه عمداً ساخته نشد

بخشی از تحلیل نیازمندی‌ها، تشخیص نیازمندی‌هایی است که نباید برآورده شوند. سه قابلیت که در نگاه اول بدیهی به نظر می‌رسند از دامنه کار خارج شده‌اند و دلیل هر یک ثبت شده است.

سه قابلیت که عمده‌اً ساخته نشدند

۱. **رزرو آنلاین.** رزرو تنها وقتی معنا دارد که سامانه بداند سازه در بازه خواسته شده آزاد است؛ این دانش نزد مالک است. فرم رزرو بدون آن یعنی تعهدی بی‌پشتوانه. سامانه کاربر را تا لحظه تماس می‌رساند و همان‌جا متوقف می‌شود.

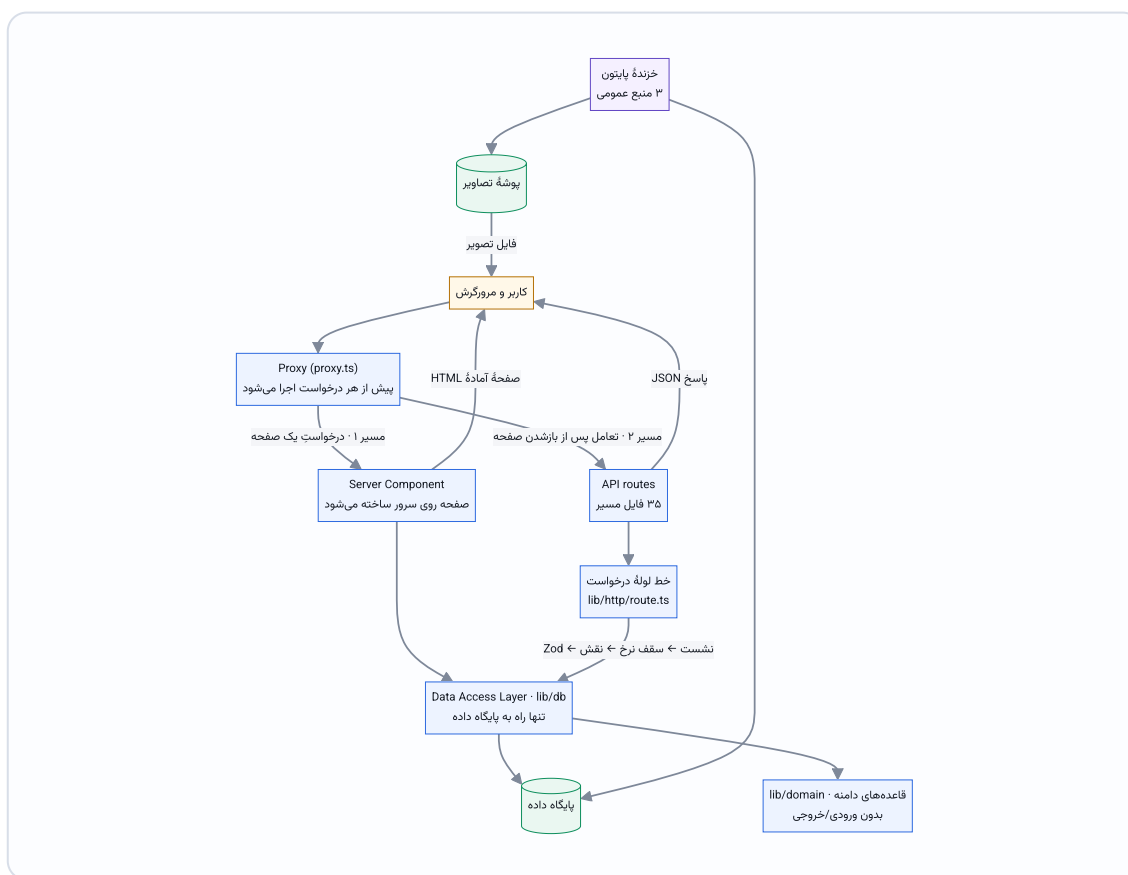
۲. **درگاه پرداخت.** اتصال به درگاه بانکی نیازمند نماد اعتماد الکترونیکی و شخصیت حقوقی است. به جای شبیه‌سازی یک درگاه جعلی که هیچ چیز واقعی را نمی‌آزماید، پلن ویژه با تأیید دستی کارشناس پیاده شده است: منطق کسب‌وکار واقعی است و فقط لحظه تراکنش بیرون از سامانه رخ می‌دهد.

۳. **سرویس‌های میزبانی‌شده.** ابزارهای رصد خطا، تحلیل رفتار کاربر و احراز هویت به‌عنوان سرویس، یا پولی‌اند یا برای ایران در دسترس نیستند. معادل درون‌کدی آنچه لازم بود (ثبت ساخت‌یافته رخدادها و شناسه خطا) ساخته و بقیه کنار گذاشته شده است.

طراحی سامانه

۱-۴ معماری کلی و دو مسیر داده

کل سامانه (رابط کاربری، مسیرهای برنامه‌نویسی و رندر سمت سرور) در یک پایگاه کد قرار دارد و به صورت یک واحد مستقر می‌شود. این یک چارچوب headless نیست؛ تفاوتی که پیامد معماری مهمی دارد و در ادامه توضیح داده می‌شود.



شکل ۴-۱ - معماری کلی سامانه. صرف‌نظر از اینکه درخواست از کدام مسیر بیاید، در نهایت از یک Data Access Layer (لایه واحد دسترسی به داده) رد می‌شود؛ Proxy پیش از هر چیز روی مسیر است، و هر درخواست /api/ پیش از رسیدن به لایه داده از یک خط لوله واحد می‌گذرد.

۱-۱-۴ دو مسیر، یک لایه داده

مسیر یکم - سرور خودش از لایه داده می‌خواند. صفحه اول هر نشانی سامانه - از صفحه نخست، کاتالوگ و جزئیات رسانه تا تحلیل بازار، پیشخوان کاربر و تک‌تک بخش‌های پنل مدیریت - در یک Server Component ساخته می‌شود؛ کدی که فقط روی سرور اجرا می‌شود و کدش هرگز به مرورگر نمی‌رود. سرور هنگام ساختن صفحه، خودش تابع لایه داده را صدا می‌زند و HTML آماده را همراه داده‌اش برمی‌گرداند؛ فقط یک پرش، از سرور به پایگاه داده. در کاتالوگ، پالایه‌ها از خود نشانی صفحه خوانده می‌شوند، پس هر نمای پالایه‌شده یک نشانی واقعی دارد که هم قابل اشتراک‌گذاری است و هم برای موتور جست‌وجو خواندنی.

مسیر دوم – مرورگر از یک آدرس `/api/` می‌خواند. فقط کاری که کاربر پس از بازشدن صفحه انجام می‌دهد (ثبت‌نام و ورود، ثبت آگهی، ثبت دیدگاه، گرفتن شماره تماس صاحب رسانه، تصمیم کارشناس، ورق‌زدن جدول‌های پزل) با `fetch` به یکی از مسیرهای `/api/` می‌رود و پاسخ JSON می‌گیرد. همین مسیر، واسطه عمومی سامانه برای مصرف‌کننده بیرونی هم هست.

قاعده کوتاه است: **صفحه اول همیشه همراه داده‌اش روی سرور ساخته می‌شود؛ مرورگر فقط در پاسخ به کاری که کاربر انجام می‌دهد از `/api/` می‌پرسد.**

نکته کلیدی این است که هر دو مسیر یک لایه داده را صدا می‌زنند. پس منطق پرس‌وجو یک بار نوشته شده، در دو جا تکرار نشده، و هیچ‌گاه پیش نمی‌آید که صفحه یک چیز نشان دهد و آدرس `/api/` چیز دیگری.

چرا هر صفحه‌ای مسیر برنامه‌نویسی خودش را صدا نمی‌زند؟

این نخستین پرسشی است که یک سامانه Next.js برمی‌انگیزد، و پاسخ با یک تشبیه روشن می‌شود. یک واسطه برنامه‌نویسی headless مانند آشپزخانه ابری است: فقط ارسال بیرون، بدون سالن پذیرایی. هر بشقاب از دریچه تحویل بیرون می‌رود، چون راه دیگری وجود ندارد – این خاصیت ساختمان است، نه نشانه کیفیت.

این سامانه یک **رستوران با سرویس بیرون‌بر** است. مهمانی که سر میز نشسته (صفحه‌ای که در حال ساخته شدن است) غذایش را مستقیم از آشپزخانه می‌گیرد؛ بدون بسته‌بندی و بدون پیک – این مسیر یکم است. سفارش بیرون‌بر (مرورگری که پس از بارگذاری داده تازه می‌خواهد) بسته‌بندی می‌شود و پیک می‌بردش – این مسیر دوم است. آشپزخانه (`lib/db/`) برای هر دو یکی است و تنها اتاقی است که به انبار (پایگاه داده) راه دارد. **دستور پخت‌ها** (`lib/domain/`) جدا از آشپزخانه نوشته شده‌اند تا بی‌روشن کردن اجاق واری شوند، و **در ورودی** (خط لوله `lib/http/route.ts`) هر سفارش بیرون‌بر را همیشه به یک ترتیب بررسی می‌کند: چه کسی سفارش داده، آیا زیادی سفارش می‌دهد، آیا اجازه این غذا را دارد، و آیا سفارش معنا دارد – و فقط آن وقت آن را به آشپزخانه می‌دهد.

فرستادن بشقاب مهمان سر میز از دریچه تحویل به بیرون و برگرداندنش به داخل، فقط به‌خاطر یکدستی، کندتر و بی‌فایده است – و دقیقاً همان کاری است که عبور دادن رندر هر صفحه از `fetch("/api/...")` به آن می‌انجامد. مسیر یکم نه یک میان‌بر است و نه یک نقص؛ الگوی توصیه‌شده خود چارچوب و گزینه سریع‌تر است.

تفاوت عملی چشمگیر است: مسیر یکم یک پرسش شبکه دارد (سرور به پایگاه داده) و حدود ۵ تا ۲۰ میلی‌ثانیه هزینه می‌برد، در حالی که صفحه‌ای که مسیر برنامه‌نویسی خودش را صدا بزند دست‌کم دو پرسش دارد، باید درخواست بسازد و JSON را تولید و دوباره تجزیه کند، و حدود ۴۰ تا ۱۵۰ میلی‌ثانیه می‌گیرد. افزون بر آن، تولید ایستا و انبارسازی در مسیر یکم کار می‌کند و در حالت دوم می‌شکند، چون به نشانی مطلق و سرور در حال اجرا نیاز دارد.

۲-۴ موتور پایگاه داده و مدل داده

موتور انتخاب‌شده SQLite است؛ نقطه‌ای که معمولاً نخستین فشار داور روی آن وارد می‌شود، پس استدلالش صریح می‌آید. بار این سامانه خواندن‌محور است (کاربران جست‌وجو و مطالعه می‌کنند و نوشتن رویدادی کم‌بسامد است) و روی یک نمونه اجرایی مستقر می‌شود. در این شرایط SQLite در حالت WAL خواندن هم‌زمان را بدون قفل‌شدن پشتیبانی می‌کند و یک سرویس جداگانه، یک استخر اتصال و یک نقطه خرابی دیگر را از معماری حذف می‌کند.

این انتخاب بن‌بست هم نیست، و این ادعا آزموده شده است نه فرض‌شده. چون همه دسترسی به داده از یک ORM (لایه‌ای که به جای SQL دستی کار می‌کند) رد می‌شود و هیچ پرس‌وجوی خام SQL جای‌جای کد پراکنده نیست، موتور فقط از روی رشته اتصال خوانده می‌شود و مهاجرت به PostgreSQL یک فرمان است: خواندن همه جدول‌ها، ساختن آن‌ها روی مقصد، کپی والد‌ها پیش از فرزندان، و تطبیق شمارش دو طرف پیش از اعلام موفقیت. در یک اجرای واقعی روی PostgreSQL 16، هر ۳,۵۶۲ ردیف منتقل شد، شمارش هر ده جدول خواند، و سامانه روی موتور تازه ساخته و سرو شد و به همان پرسش‌ها همان پاسخ‌ها را داد.

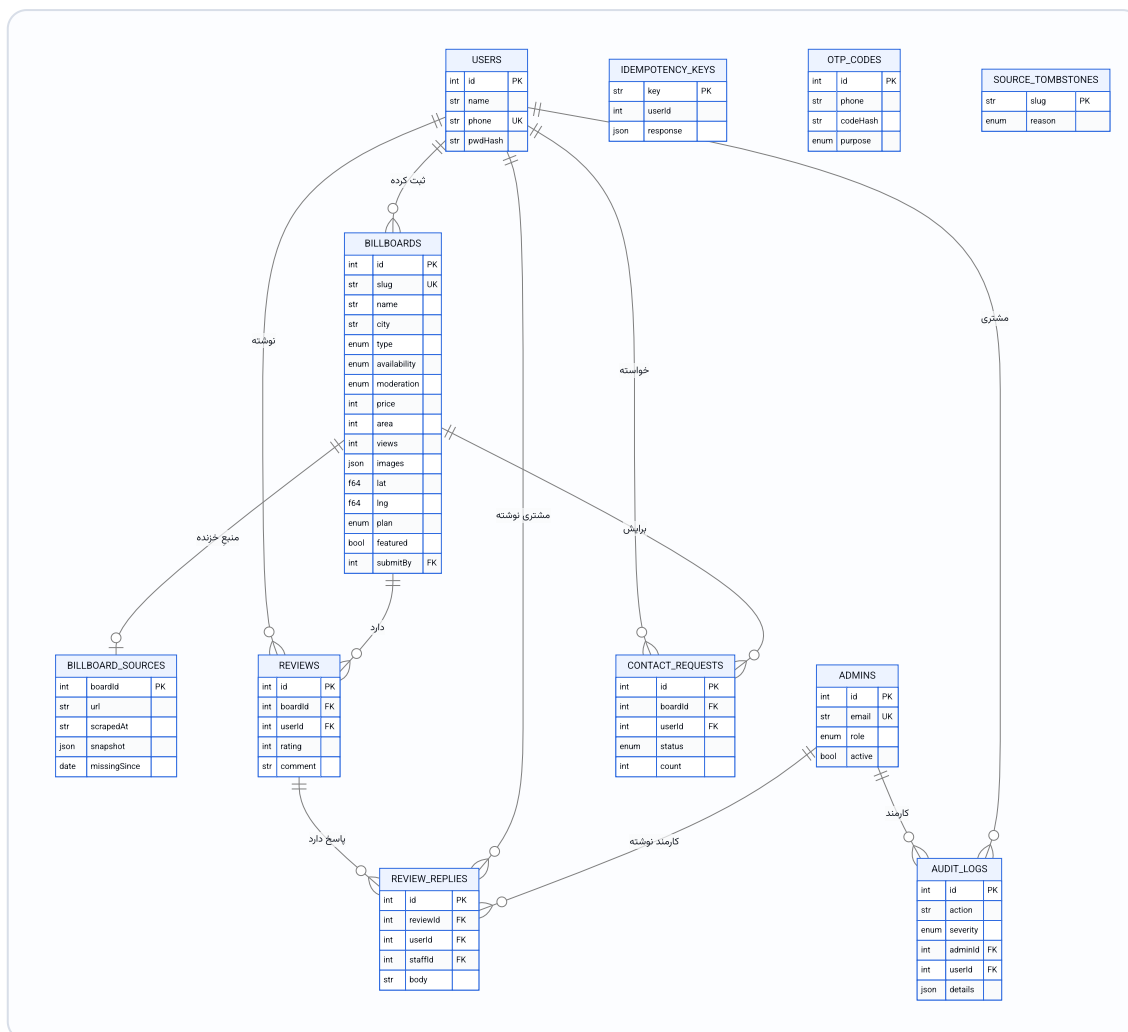
در سراسر کد تنها دو نقطه به موتور وابسته‌اند و هر دو صریح‌اند: خواندن از داخل ستون JSON، که دو موتور آن را متفاوت می‌نویسند؛ و ستون متنی یکسان‌شده جست‌وجو، که در SQLite با trigger و در PostgreSQL به صورت ستون محاسبه‌شده نگه داشته می‌شود. این ستون همان متن نام، شهر، نشانی و آژانس است با «ی» و «ك» عربی به «ی» و «ك» فارسی، نیم‌فاصله به فاصله و رقم فارسی به لاتین؛ پیش از آن، جست‌وجوی «کرج» با صفحه‌کلید عربی از ۲۷۳ رسانه کرج تنها یکی را می‌یافت. آنچه امروز از دست می‌رود، نوشتن هم‌زمان از چند نمونه اجرایی سرور است که این سامانه به آن نیازی ندارد.

مدل داده در پنج عدد خلاصه می‌شود: ۱۲ جدول (در نمودار زیر «موجودیت»)، ۹ نوع شمارشی (enum) برای هر مجموعه بسته از مقدارها، ۳۳ نمایه و قید یکتایی که پرس‌وجوهای پرتکرار را سریع و داده را از تکرار ناخواسته حفظ می‌کنند، ۱۷ مهاجرت که تاریخچه نسخه‌به‌نسخه همین طرح‌واره است، و ۳,۵۳۶ ردیف رسانه در جدول اصلی.



ستون مرکزی مدل، جدول `billboards` است. اگر رسانه از راه فرم ثبت آگهی وارد شده باشد به کاربر ثبت‌کننده‌اش پیوند می‌خورد، و اگر خزنده آن را آورده باشد یک ردیف همراه در `billboard_sources` دارد که دفتر کار خزنده است (نشانی منبع، زمان خواندن، و آخرین چیزی که منبع گفته بود). نظرها، پاسخ‌ها و درخواست‌های تماس همگی به همین جدول وصل می‌شوند. کاربران عادی (`users`) و کارکنان (`admins`) دو جدول جدا دارند، ولی هر ارجاع به هر کدام یک کلید خارجی واقعی است – حتی نویسنده یک پاسخ و انجام‌دهنده یک رخداد ممیزی. سه جدول باقی‌مانده – کلید یکتایی عملیات، رمزهای یک‌بارمصرف و «سنگ قبر» ردیف‌هایی که عمداً حذف شده‌اند – به کسب‌وکار ربطی ندارند و فقط زیرساخت صحت‌اند.

دو پرسش، دو ستون. وضعیت یک رسانه در ابتدا یک ستون متنی `status` بود که به دو پرسش مستقل هم‌زمان جواب می‌داد: «سازه الان آزاد است؟» و «آگهی از بررسی گذشته است؟». نتیجه این بود که هر پرس‌وجوی عمومی باید فهرستی از چهار وضعیت را کنار می‌گذاشت و فرم ویرایش مدیر می‌توانست آگهی را به «در انتظار» برگرداند و ماشین حالت بررسی را دور بزند. اکنون دو ستون شمارشی داریم: `availability` (آزاد، مشغول، رزرو، غیرفعال) و `moderation` (در انتظار، در انتظار پرداخت، نیاز به اصلاح، رد، منتشر). «منتشرشده» یعنی `moderation = approved` و فقط یک جا تعریف شده است؛ و وضعیت بررسی تنها از مسیر تصمیم کارشناس تغییر می‌کند.



شکل ۴-۲ - نمودار موجودیت-رابطه (ERD): هر جعبه یک جدول است و هر خط یک پیوند کلید خارجی میان دو جدول. سه جدول - `SOURCE_TOMBSTONES` و `OTP_CODES`، `IDEMPOTENCY_KEYS` - به هیچ جدول دیگری وصل نیستند، چون به کسب‌وکار ربطی ندارند و فقط زیرساخت صحت‌اند. رد ممیزی، برعکس، انجام‌دهنده را با کلید خارجی به جدول خودش (کارمندی یا مشتری) وصل می‌کند.

۱-۲-۴ نمایه‌گذاری بر پایه پرس‌وجوهای واقعی

نمایه‌ها بر اساس شکل پرس‌وجوهایی که رابط کاربری واقعاً می‌سازد تعریف شده‌اند، نه به صورت حدسی روی تک‌تک ستون‌ها. سه نمایه ترکیبی `(city, moderation)`، `(city, type)` و `(type, price)` دقیقاً پالایه‌های صفحه جست‌وجو را پوشش می‌دهند. افزون بر آن، دو کلید مرتب‌سازی پرکاربرد در جدول ذخیره شده‌اند تا مرتب‌سازی به‌جای محاسبه در حافظه، در خود پایگاه داده انجام شود.

۳-۴ طراحی واسط برنامه‌نویسی

سامانه ۳۵ فایل مسیر دارد و روی هم ۴۴ نقطه پایانی عرضه می‌کند. تفاوتشان این است: هر فایل مسیر به یک نشانی وصل است (مثلاً `/api/billboards`)، اما یک فایل می‌تواند چند فعل HTTP را جداگانه هندل کند - همان نشانی برای `GET` «فهرست را بده» و برای `POST` «یکی اضافه کن». پس ۴۴ نقطه پایانی یعنی ۴۴ ترکیب نشانی و فعل. طراحی از قرارداد REST پیروی می‌کند: «چه چیزی» در نشانی می‌آید و «چه کاری» در فعل.

جدول ۴-۱ - توزیع نقاط پایانی بر حسب فعل HTTP

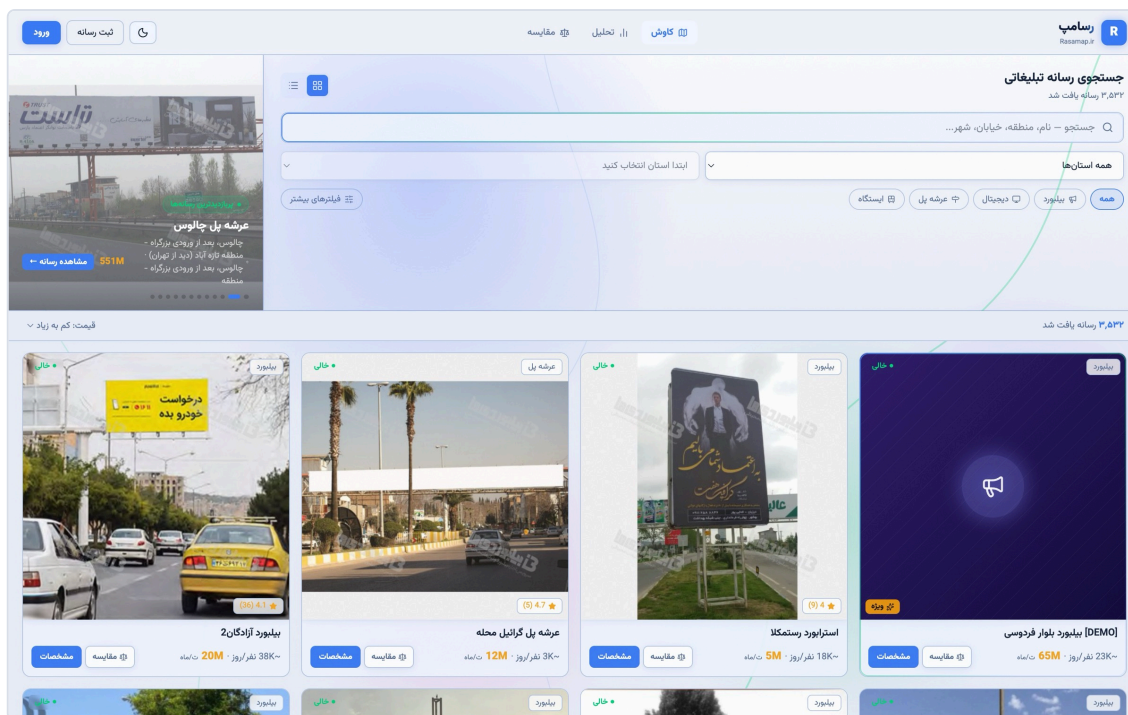
فعل	تعداد	کاربرد
GET	۱۸	خواندن: فهرست رسانه‌ها، جزئیات، آمار، نظرها، داده‌های پنل مدیریت، و بررسی زنده‌بودن سامانه
POST	۱۵	ایجاد و کنش: ورود، ثبت‌نام، ثبت آگهی، درخواست تماس، ارسال رمز یک‌بارمصرف
PATCH	۶	تغییر جزئی: ویرایش رسانه یا سرخ، تغییر نقش مدیر، تغییر رمز خود
DELETE	۳	حذف: رسانه، نظر، پاسخ
PUT	۲	جایگزینی کامل رکورد در پنل مدیریت
مجموع	۴۴	در ۳۵ فایل مسیر

یک تصمیم طراحی در این جدول پنهان است: دریافت شماره تماس با POST انجام می‌شود، نه GET. از نظر معنایی هم درست است (این عمل یک رکورد درخواست ایجاد می‌کند) و از نظر عملی، خواندن انبوه با یک حلقه ساده روی نشانی‌ها را ناممکن می‌کند.

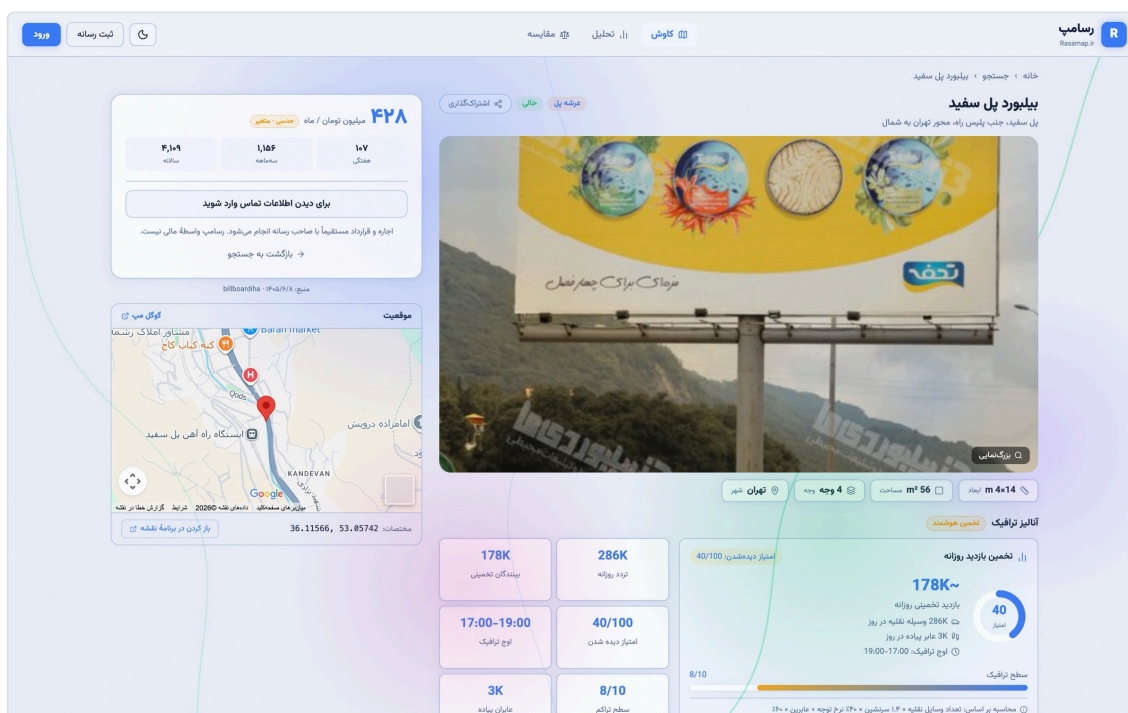
۴-۴ طراحی رابط کاربری

رابط کاربری کاملاً فارسی و راست‌به‌چپ است. این تصمیم فراتر از افزودن یک صفت جهت است: چیدمان، جهت نمادها، ترتیب ستون‌های جدول و حتی جهت انیمیشن‌ها بر همان اساس تعریف شده‌اند.

سبک‌دهی به‌صورت شیء درون‌خطی روی هر مؤلفه انجام می‌شود. این انتخاب خطوط کد را طولانی‌تر می‌کند، ولی سامانه طراحی را از وابستگی به پیکربندی پالایش یک ابزار بیرونی جدا نگه می‌دارد و تضمین می‌کند سبکی که در کد دیده می‌شود همان است که در مرورگر اجرا می‌شود.



شکل ۳-۴ - صفحه جستجو و پالایش. پالایه‌های شهر، نوع، قیمت و ابعاد بالای شبکه نتایج؛ هر کارت قیمت ماهانه و برآورد بازدید روزانه را نشان می‌دهد.



شکل ۴-۴ - صفحه جزئیات رسانه: گالری تصویر، مشخصات فنی، موقعیت روی نقشه، و پنل «آنالیز ترافیک» که برآورد بازدید روزانه و مبنای محاسبه‌اش را می‌آورد. شماره تماس در این صفحه نیست؛ جایش دکمه «برای دیدن اطلاعات تماس وارد شوید» است.

سه اصل در طراحی حالت‌های رابط رعایت شده است: هر فهرست حالت «خالی» طراحی‌شده دارد و صفحه سفید نشان نمی‌دهد؛ هر دکمه ارسال هنگام پرواز درخواست غیرفعال می‌شود تا ارسال دوباره رخ ندهد؛ و هر خطا پیام فارسی روشن دارد، نه کد وضعیت خام.

۵-۴ سازمان‌دهی کد و نقشه وابستگی‌ها

پروژه ۲۰۲ فایل TypeScript دارد. وقتی فایلی چیزی را از فایل دیگر همین پروژه `import` می‌کند، یک «وابستگی» میانشان ساخته می‌شود؛ در کل پروژه ۵۲۰ چنین وابستگی وجود دارد. این عددها با یک اسکریپت که خود مخزن را می‌خواند و همه دستوره‌های `import` را می‌شمارد استخراج شده‌اند، نه با تخمین. پس آنچه در ادامه می‌آید تصویر معماری واقعی موجود است، نه معماری آرزوشده.

جدول ۲-۴ – لایه‌های کد، تعداد فایل و نقش هر لایه

لایه	فایل	نقش
Proxy	۱	<code>proxy.ts</code> – پیش از هر درخواست اجرا می‌شود
صفحه‌ها	۵۳	آنچه کاربر در مرورگر می‌بیند، به همراه چیدمان، مرز خطا و حالت بارگذاری هر بخش
مؤلفه‌های رابط	۴۲	قطعه‌های رابط کاربری که چند صفحه از آن‌ها استفاده می‌کنند
API routes	۳۵	نقطه‌های ورود HTTP؛ هر کدام فقط قراردادش را اعلام می‌کند
خط لوله درخواست	۴	<code>lib/http/</code> – ترتیب ثابت بررسی‌ها، پاسخ‌های مشترک، پیام‌های فارسی اعتبارسنجی
احراز هویت	۷	نشست، بازیگر نوع‌دار (مشتری/کارمند)، گذرواژه، سقف نرخ، رد ممیزی
Data Access Layer	۱۸	<code>lib/db/</code> – تنها راه رسیدن به پایگاه داده، یک ماژول برای هر منبع
قاعده‌های دامنه	۶	<code>lib/domain/</code> – قاعده‌های محصول بدون هیچ ورودی/خروجی
ابزارهای مشترک	۱۸	ثبت رخداد، قالب‌بندی، جغرافیا، ابزارهای سمت مرورگر
طرح‌واره و داده اولیه	۷	پرکردن اولیه، واردکننده شبانه، حذف تکراری، افزودن مختصات
پیکربندی	۴	تنظیمات ساخت و اجرا

صفحه‌ها و مسیرهای برنامه‌نویسی — چارچوب از روی نام پوشه نشانی می‌سازد

- `page.tsx` Server Component — صفحه نخست
- `explore/page.tsx` Server Component — جست‌وجو
- `billboard/[slug]/page.tsx` Server Component — جزئیات
- `admin/(panel)/` پنل مدیریت — هر بخش نشانی خودش؛ حساب کارمند روی سرور بررسی می‌شود
- `api/` `route.ts` فایل ۳۵ — هر کدام با `defineRoute` اعلام می‌کند چه می‌خواهد
 - `billboards/route.ts` فهرست و پالایش
 - `auth/` ورود، ثبت‌نام، رمز یک‌بارمصرف
 - `admin/` مسیرهای مدیریتی

`components/` ۴۲ قطعه رابط کاربری قابل استفاده مجدد

`lib/` منطق مشترک — هیچ صفحه یا مسیری مستقیم به پایگاه داده دست نمی‌زند

- `http/route.ts` خط لوله ثابت هر درخواست
- `db/listings.ts`، `client.ts`، `billboards`، `reviews.ts`، `leads.ts`، `customers.ts`، `staff.ts` و ... تنها لایه‌ای که پایگاه داده را می‌شناسد؛ `client.ts`، `billboards` (خواندن و نوشتن جدا)، `reviews.ts`، `leads.ts`، `customers.ts`، `staff.ts`
- `domain/` قاعده‌های خالص محصول، با آزمون واحد
- `auth/` نشست، بازیگر نوع‌دار، گذرواژه
- `rate-limit/` سقف‌های نام‌دار؛ شمارنده در حافظه یا Redis
- `client/` ابزارهایی که فقط در مرورگر اجرا می‌شوند
- `types.ts` انواع داده، بدون خود داده

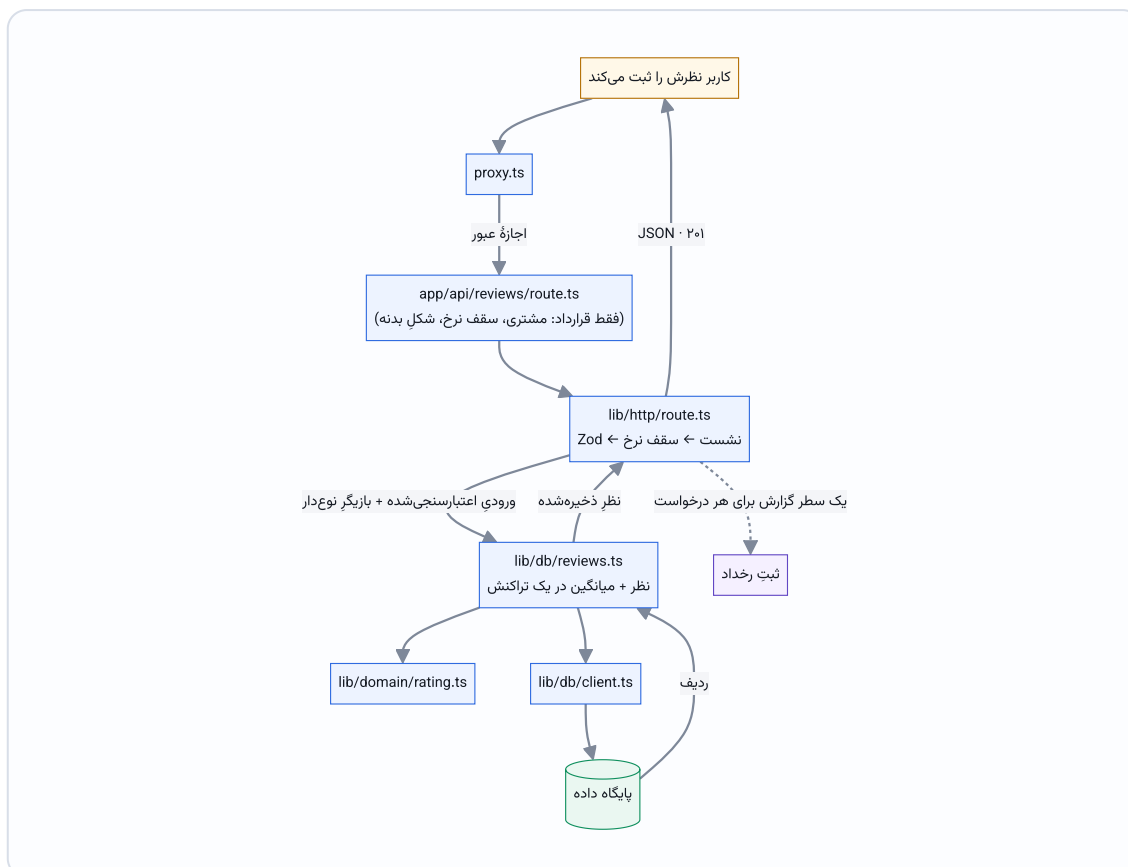
`prisma/schema.prisma` ۱۲ جدول، ۹ نوع شمارشی، ۳۳ نمایه، ۲۰ مهاجرت

`proxy.ts` Proxy — پیش از هر درخواستی اجرا می‌شود

شکل ۴-۵ — ساختار پوشه‌های پروژه. نکته کلیدی این چیدمان این است که `lib/db/` تنها راه رسیدن به پایگاه داده است — و این فقط یک قرار نیست: قاعده ESLint واردکردن اتصال پایگاه داده را بیرون از این پوشه، و هر واردکردن دارای ورودی/خروجی را در `lib/domain/`، به‌عنوان خطا رد می‌کند.

۴-۵-۱ سفر یک درخواست، با نام فایل‌های واقعی

نمودار زیر نشان می‌دهد وقتی کاربر زیر یک رسانه نظر ثبت می‌کند، درخواست او گام‌به‌گام از چه مسیری می‌گذرد. هر جعبه یک فایل واقعی در مخزن است و هر پیکان یک فراخوانی یا یک پاسخ. پیکان نقطه‌چین کاری است که در حاشیه انجام می‌شود و پاسخ کاربر منتظرش نمی‌ماند — اینجا، نوشتن یک سطر گزارش برای هر درخواست.



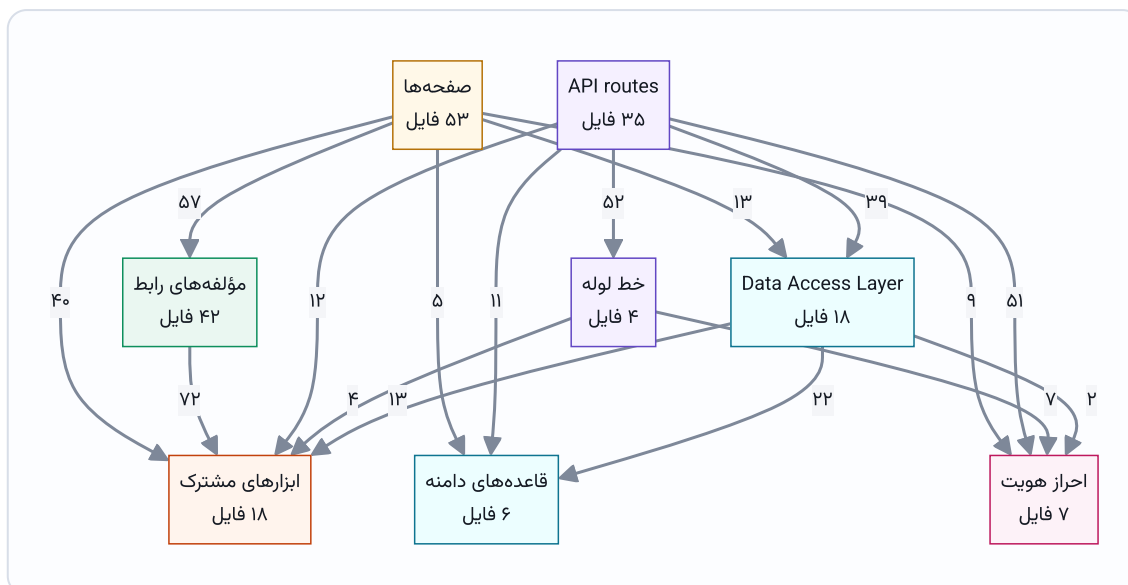
شکل ۴-۶ - سفر یک درخواست، از کلیک کاربر تا پایگاه داده و برگشت. فایل مسیر هیچ منطقی ندارد؛ خط لوله بررسی‌ها را به ترتیب ثابت اجرا می‌کند و فقط ورودی سالم را به لایه داده می‌دهد. صفحه‌ها همین لایه داده را بی‌واسطه صدا می‌زنند (مسیر یکم) و از خط لوله نمی‌گذرند، چون درخواست HTTP نیستند.

۶-۴ گراف وابستگی پروژه

دو شکل بعدی از یک داده واحد ساخته شده‌اند - همان شمارشی `import` ها - ولی از دو فاصله. نخستین شکل کل پروژه را در حد «لایه» جمع می‌بندد؛ دومی تک‌تک فایل‌ها را نشان می‌دهد.

۱-۶-۴ وابستگی میان لایه‌ها

هر فایل پروژه به یکی از این لایه‌ها تعلق دارد. اگر همه `import` ها را بر حسب لایه فایل مبدأ و لایه فایل مقصد دسته‌بندی کنیم، این نمودار به دست می‌آید. هر پیکان از لایه A به لایه B یعنی «فایل‌های A چیزهایی را از فایل‌های B وارد می‌کنند»، و عدد رویش می‌گوید این کار در کل کد چند بار تکرار شده. برای نمونه، «`→ API routes` خط لوله : ۵۲ `import`» یعنی در ۵۲ نقطه، یک فایل مسیر چیزی را از خط لوله درخواست وارد کرده است. (برای خوانایی، فقط پیکان‌هایی رسم شده‌اند که میان دو لایه متفاوت از این هشت لایه‌اند و دست‌کم چهار بار تکرار شده‌اند؛ پیوندهای درون‌لایه‌ای و لایه‌های کوچک‌تر کنار گذاشته شده‌اند.)



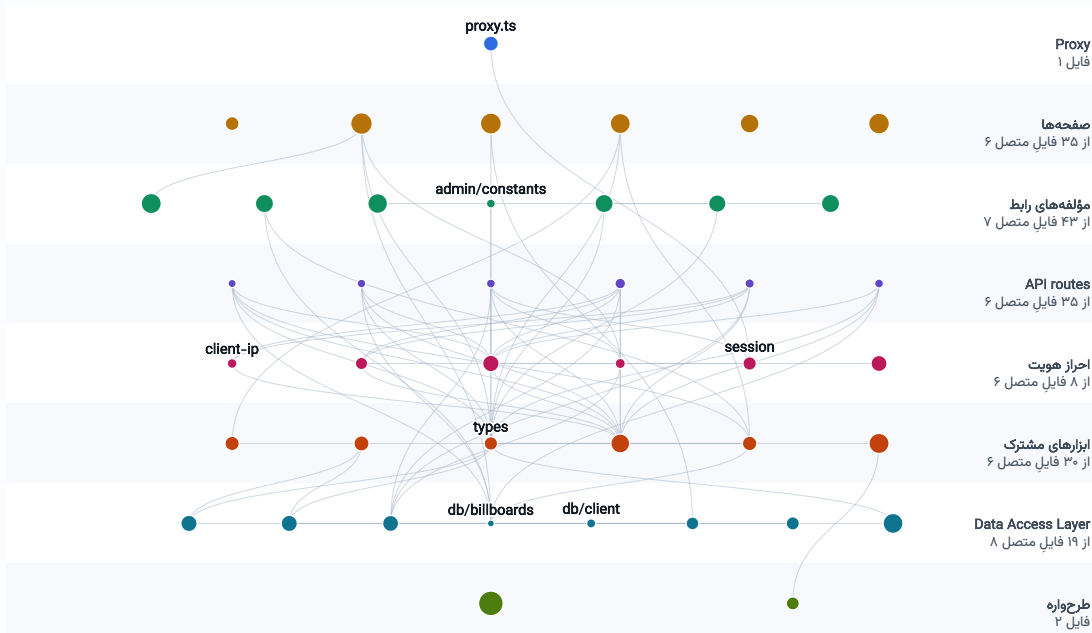
شکل ۴-۷ - وابستگی میان لایه‌های کد؛ عدد هر پیکان تعداد `import` است. هر ۳۵ مسیر به خط لوله وصل‌اند (۵۲ پیکان) و بیشتر کار احراز هویت را از آن می‌گیرند. نکته مهم‌تر آن است که هیچ پیکانی از مسیرها یا صفحه‌ها به خود پایگاه داده نمی‌رود؛ هر دو فقط به `Data Access Layer` وصل‌اند - مسیرها از راه خط لوله، و صفحه‌ها بی‌واسطه (مسیر یکم بخش ۴-۱). لایه دامنه هم هیچ پیکان خروجی‌ای ندارد؛ به چیزی جز خودش وابسته نیست.

۴-۶-۲ گراف کامل، فایل به فایل

شکل بعدی همان داده است، ولی این بار بدون جمع‌بندی: تک‌تک فایل‌ها دیده می‌شوند. هر نوار افقی یک لایه است (نامش در سمت راست)، هر دایره یک فایل، و بزرگی دایره به تعداد خطوط آن فایل. در هر نوار، فایل‌ها بر پایه «چند فایل دیگر واردشان می‌کنند» چیده شده‌اند: هرچه فایل وابسته بیشتری داشته باشد به وسط نوار نزدیک‌تر است. پس دایره‌های درشت نزدیک به مرکز هر نوار، همان فایل‌هایی‌اند که بیشتر پروژه به آن‌ها تکیه دارد.

از آن ۲۰۲ فایل، ۲۹ فایل نه چیزی `import` می‌کنند و نه جایی وارد می‌شوند - فایل‌های قراردادی چارچوب (`layout`، `loading`، `error`) که خود `Next.js` صدایشان می‌زند - پس ۱۷۳ فایل متصل می‌ماند، با ۵۲۰ پیوند میانشان. کشیدن همه‌شان روی کاغذ فقط یک کلاف می‌شود؛ تنه‌ی این شلوغی ۳۵ فایل مسیر است که هرکدام خط لوله، یک سقف نرخ و یک مازول لایه داده را وارد می‌کنند. برای خواندن‌ماندن، شکل زیر از هر لایه فقط چند فایل بزرگ‌ترش را نشان می‌دهد، به‌اضافه فایل‌های نام‌گذاری شده - روی هم ۴۲ فایل و ۷۴ پیوند. الگوی کلی همان است؛ فقط انبوهگی حذف شده.

می‌کنند import هر دایره یک فایل است؛ هرچه فایل خط بیشتری داشته باشد دایره‌اش بزرگ‌تر است، و هرچه به مرکز نوارش نزدیک‌تر باشد فایل‌های بیشتری آن را



شکل ۸-۴ – گراف وابستگی در سطح فایل – نمایی خلاصه: از هر لایه چند فایل بزرگ‌تر به‌اضافه فایل‌های نام‌گذاری‌شده (۴۲ فایل از ۱۷۳ فایل متصل، ۷۴ پیوند). آنچه باید دید، تراکم دایره‌ها نزدیک مرکز سه نوار پایینی – احراز هویت، ابزارهای مشترک و Data Access Layer – است: هرچه فایلی به مرکز نوارش نزدیک‌تر باشد، فایل‌های بیشتری واردش می‌کنند. همین تصویر است که جدول بعدی به‌صورت عدد نشان می‌دهد.



نمای کلی پنل مدیریت

شکل ۹-۴ – نمای کلی پنل مدیریت. اعداد این صفحه مستقیم از پایگاه داده خوانده می‌شوند و همان‌هایی‌اند که در سراسر این نوشته نقل شده‌اند: ۳,۵۳۶ رسانه، ۳,۰۲۰ دارای مختصات (۸۵٪)، و سهم هر منبع.

۳-۶-۴ دو سنجۀ عینی از سلامتِ وابستگی‌ها

«معماری تمیز» صفتی است که هر پروژه‌ای درباره‌ی خودش می‌گوید. برای آنکه اینجا ادعا نماند، دو ویژگی گراف مستقیم از همان داده‌ی `import` سنجیده شده است.

جهت وابستگی یک‌طرفه است. لایه‌ها ترتیبی دارند – صفحه‌ها و مسیرها بالا، مؤلفه‌های رابط زیر آنها، ابزارهای مشترک و لایۀ داده پایین – و وابستگی فقط رو به پایین مجاز است. اگر لایۀ داده روزی چیزی از یک صفحه `import` کند، دو لایه به هم گره می‌خورند و هیچ‌کدام دیگر جداگانه قابل فهم یا جابه‌جایی نیستند. شمارش چنین یال‌هایی در کل پروژه صفر است.

هیچ حلقه‌ای در گراف نیست. حلقه یعنی فایل الف به ب وابسته باشد و ب – مستقیم یا با چند واسطه – دوباره به الف. کد حلقه‌دار را نمی‌توان «از اول» خواند، ترتیب مقداردهی اولیه‌اش به جزئیات بسته‌بند چارچوب وابسته می‌شود، و جدا کردن یک تکه از آن برای آزمون ناممکن است. جست‌وجوی عمق‌اول کل ۵۲۰ یال صفر حلقه پیدا می‌کند.

تنها یک در به پایگاه داده باز است. اتصال پایگاه داده را ۱۳ فایل وارد می‌کنند و همه آن‌ها درون `lib/db/` اند؛ پیش از بازبینی معماری (بخش ۳-۶) این عدد ۳۰ بود و خود مسیرها را هم شامل می‌شد. این سنجۀ برخلاف دو سنجۀ قبلی فقط اندازه‌گیری نمی‌شود، اجرا هم می‌شود: قاعدۀ ESLint واردکردن چهاردهم را رد می‌کند.

پس ساختار پوشه‌ها بازتاب ساختار نشانی‌هاست – که قرارداد خود App Router است – و ساختار وابستگی‌ها یک گراف جهت‌دار بدون دور است. این دو با هم همان چیزی‌اند که از یک چیدمان لایه‌ای انتظار می‌رود.

۴-۶-۴ معماری در خود گراف دیده می‌شود

جدول زیر همان فایل‌های نزدیک به مرکز نوارها را فهرست می‌کند؛ ستون «وابسته» یعنی چند فایل دیگر این فایل را `import` می‌کنند. خط لوله و سقف نرخ هر کدام دقیقاً ۳۵ وابسته دارند – به تعداد فایل‌های مسیر – پس «هر مسیر از یک خط لوله می‌گذرد» فقط یک ادعا در متن نیست، در ساختار واقعی وابستگی‌ها هم دیده می‌شود.

جدول ۴-۳ – پرارجاع‌ترین فایل‌های پروژه – چند فایل به هر یک وابسته‌اند

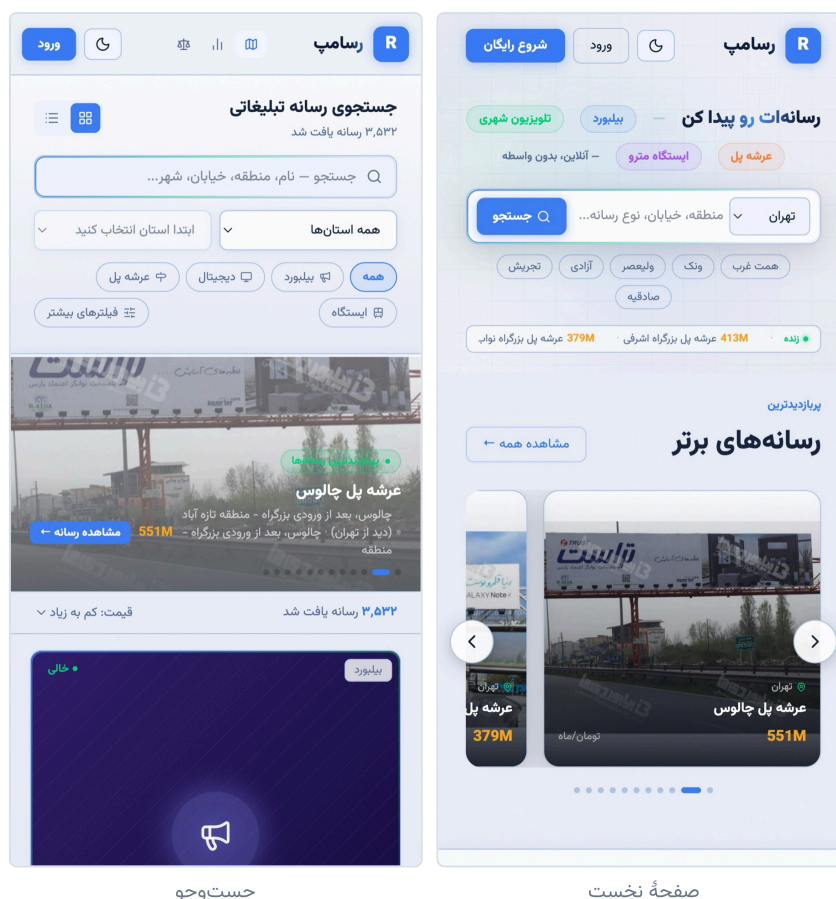
وابسته	فایل	نقش در پایپلاین
۳۷	<code>lib/types.ts</code>	انواع داده دامنۀ و برچسب‌های فارسی، بدون داده
۳۵	<code>lib/http/route.ts</code>	خط لوله: نشست ← سقف نرخ ← نقش ← اعتبارسنجی ← منطق
۳۵	<code>lib/rate-limit/index.ts</code>	سقف‌های نام‌دار نرخ درخواست
۳۱	<code>lib/format.ts</code>	قالب‌بندی عدد و قیمت به فارسی
۱۹	<code>lib/auth/actor.ts</code>	بازیگر نوع‌دار: مشتری یا کارمند
۱۷	<code>lib/http/params.ts</code>	اعتبارسنجی مشترک شناسه و صفحه‌بندی

وابسته	فایل	نقش در پایلین
۱۶	lib/db/billboards/	لایه دسترسی داده رسانه‌ها — queries.ts برای خواندن، mutations.ts برای نوشتن، core.ts برای آنچه هر دو لازم دارند
۱۳	lib/db/client.ts	تنها اتصال به پایگاه داده — همه وابسته‌ها درون lib/db/

چرا این عددها مهم‌اند

اینکه lib/db/client.ts در پایین این فهرست است، عمدی است: فقط ۱۳ فایل لایه داده به آن وابسته‌اند و هیچ صفحه یا مسیری مستقیم به پایگاه داده دست نمی‌زند — پس ادعای «یک نقطه ورود به پایگاه داده» قابل بررسی است و نه شعاری. در طرف دیگر، اینکه خط لوله و سقف نرخ درست به تعداد فایل‌های مسیر وابسته دارند یعنی هیچ مسیری از این بررسی‌ها جا نیفتاده است؛ و اگر مسیر تازه‌ای بخواهد دور خط لوله صادر شود، یک آزمون نگهبان جلویش را می‌گیرد.

یک نقشه تعاملی از همین گراف (که در آن می‌توان روی هر فایل زد و دید چه چیزی را صدا می‌زند و چه کسی صدایش می‌زند) در مخزن پروژه به نشانی docs/codemap.html قرار دارد و از روی همان استخراج خودکار ساخته می‌شود.



جست‌وجو

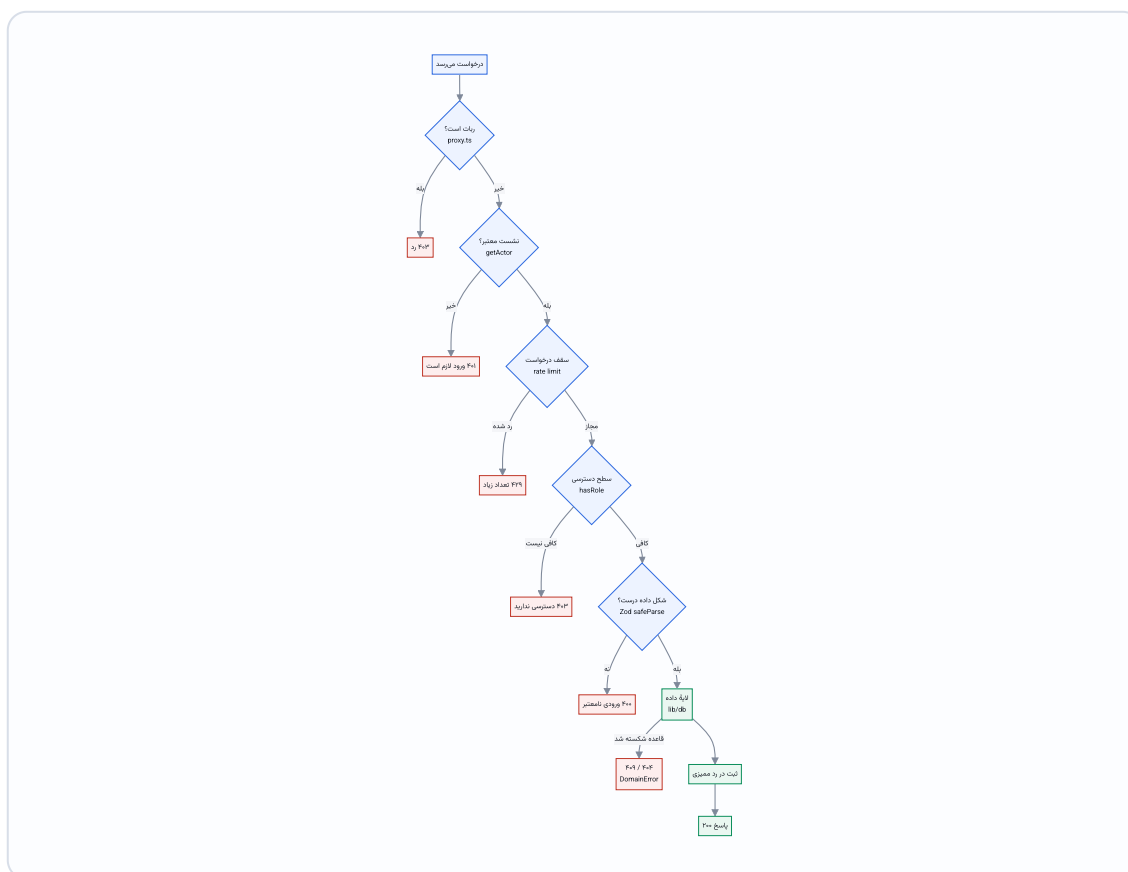
صفحه نخست

شکل ۴-۱۰ — همان رابط روی نمایشگر تلفن همراه. چیدمان، پالایه‌ها و کارت‌ها برای عرض کم بازآرایی می‌شوند، نه اینکه کوچک شوند.

پیاده‌سازی

۱-۵ پایپلاین ثابت هر درخواست

مهم‌ترین تصمیم‌پیاده‌سازی این پروژه، ثابت‌کردن ترتیب بررسی‌هاست: هر مسیر برنامه‌نویسی، پیش از آنکه به کار اصلی برسد، از دروازه‌های پشت‌سرهم یکسان می‌گذرد. این ترتیب فقط یک بار، در تابع `defineRoute` (فایل `lib/http/route.ts`)، نوشته شده است و هر ۳۵ فایل مسیر از آن استفاده می‌کنند. نمودار زیر این دروازه‌ها را نشان می‌دهد؛ هر لوزی یک بررسی است و اگر پاسخش «نه» باشد، درخواست همان‌جا با یک کد خطای HTTP رد می‌شود و جلوتر نمی‌رود (۴۰۱ = وارد نشده‌ای، ۴۰۳ = اجازه نداری، ۴۲۹ = زیادی درخواست فرستادی، ۴۰۰ = ورودیات بدشکل است).



شکل ۵-۱ - خط لوله ثابت هر درخواست. ترتیب دروازه‌ها عوض نمی‌شود؛ چرایی‌اش - که امنیتی است، نه سلیقه‌ای - درست پس از همین شکل آمده است. لایه داده هم خودش پاسخ HTTP نمی‌سازد؛ قاعده شکسته‌شده را به‌صورت `DomainError` با پیام فارسی اعلام می‌کند و خط لوله آن را به کد وضعیت ترجمه می‌کند.

```
export const POST = defineRoute(
  {
    name: "admin/listings/[id]/decision",
    access: { staff: "admin" }, // ۳ - نشست و سطح دسترسی ۱
    rateLimit: adminApiRateLimit, // ۲ - سقف نرخ (بدون آن کامپایل نمی‌شود)
    params: idParams, // ۴ - اعتبارسنجی شناسه
    body: DecisionSchema, // ۴ - اعتبارسنجی بدنه
  },
  async ({ params, body, audit }) => { // ۵ - منطق کسب‌وکار
    const { before, after } = await decideListing(params.id, body.decision, body.note,
    await audit("listing_approved", { details: { from: before.moderation, to: after.mod
    return NextResponse.json({ listing: after });
  },
);
```

قطعه کد ۱-۵ - خط لوله در عمل، از مسیر تصمیم کارشناس. مسیر فقط اعلام می‌کند چه می‌خواهد؛ زمان اجرای هر بررسی را خط لوله تعیین می‌کند، پس هیچ مسیری نمی‌تواند ترتیب را جابه‌جا کند.

دلیل ثابت‌بودن ترتیب، امنیتی است و نه سلیقه‌ای:

- اگر **سقف درخواست** پیش از **بررسی نشست** بیاید، مهاجم بدون داشتن حساب می‌تواند سهمیه درخواست را تمام کند و کاربر واقعی را از سرویس محروم سازد.
- اگر **اعتبارسنجی** پیش از **سقف درخواست** بیاید، مهاجم می‌تواند بار سنگین بفرستد و سرور را مجبور به تجزیه رایگان آن کند.

در نسخه نخست، این ترتیب در هر ۳۵ فایل مسیر جداگانه و دستی نوشته شده بود و تنها نگرهبانش یک قاعده متنی در سند قواعد پروژه بود. اکنون در یک جا نوشته شده؛ نوع `defineRoute` TypeScript مسیر بی‌سقف نرخ را نمی‌پذیرد، و یک آزمون نگرهبان هر مسیری را که دور خط لوله صادر شود رد می‌کند. همین خط لوله هر درخواست را در یک سطر گزارش ثبت می‌کند و پاسخ مسیرهای غیرعمومی را از انبارهای مشترک دور نگه می‌دارد.

۲-۵ احراز هویت و کنترل دسترسی

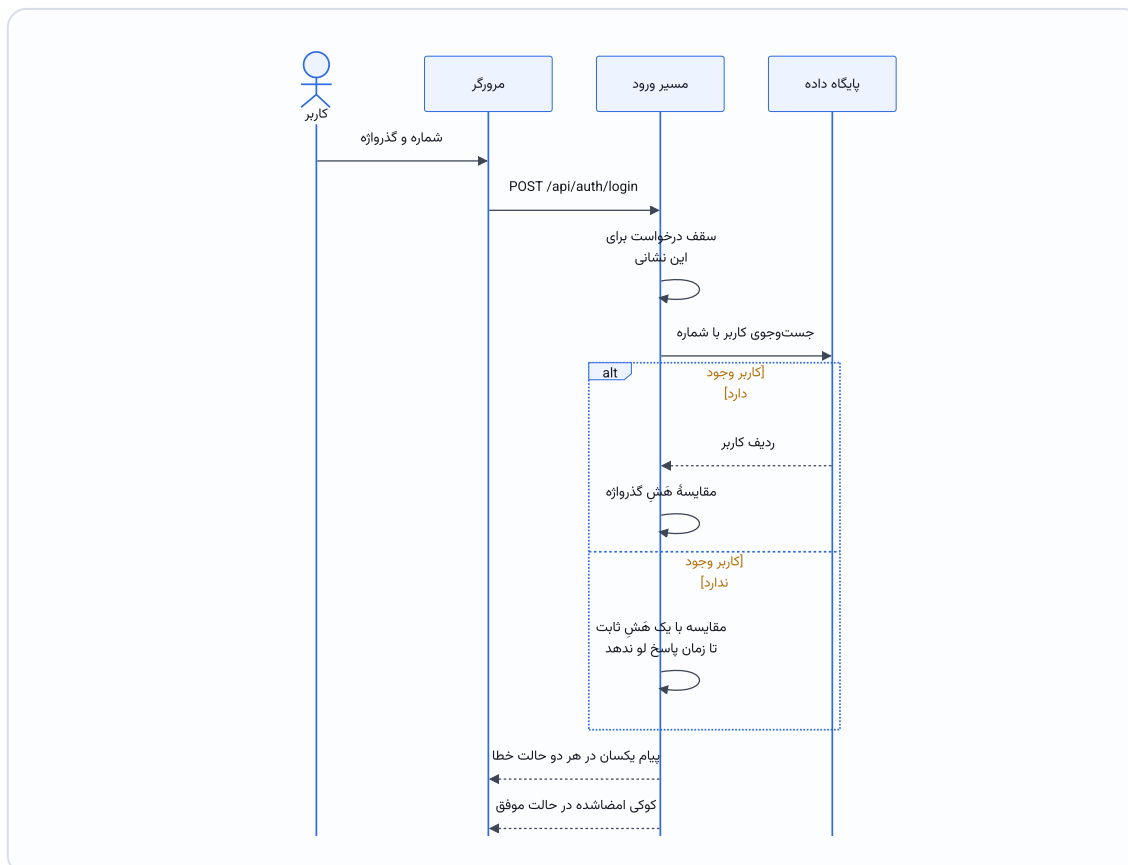
نشست کاربر یک توکن امضا شده با الگوریتم HS256 است که در کوکی `HttpOnly` با صفت `SameSite=Lax` قرار می‌گیرد؛ چنین کوکی‌ای برای کد JavaScript صفحه قابل خواندن نیست، پس اسکریپت تزریق شده هم نمی‌تواند نشست را بدزد، و مرورگر آن را همراه هیچ درخواست تغییردهنده‌ای از وب‌گاه دیگر نمی‌فرستد. توکن امضا شده به‌خودی خود تا پایان اعتبارش معتبر می‌ماند، حتی اگر صاحب حساب رمزش را عوض کند؛ پس هر درخواست توکن را با ردیف حساب در پایگاه داده می‌سنجد: یک شمارنده نسخه در حساب، با تغییر یا بازنشانی رمز و غیرفعال شدن حساب بالا می‌رود و همه نشست‌های پیشین را می‌بندد؛ خروج از حساب شناسه همان توکن را باطل می‌کند؛ و تمدید خودکار نشست هفت روز پس از ورود متوقف می‌شود. قفل شدن حساب پس از چند تلاش نادرست هم دیگر ابزار بیرون‌گذاشتن صاحب حساب نیست: مرورگری که پیش‌تر با موفقیت وارد شده، سهمیه تلاش جداگانه خودش را دارد. گذرواژه‌ها هرگز به‌صورت متن آشکار ذخیره نمی‌شوند؛ به‌جایش با `bcrypt` (با ضریب هزینه ۱۲) از هرکدام یک «هش» ساخته می‌شود — یک رشته یک‌طرفه که از روی گذرواژه به دست می‌آید ولی از روی آن نمی‌توان گذرواژه را بازساخت.

نکته ظریفتر، رفتار سامانه در برابر شماره‌ای است که در سامانه ثبت نیست. اگر سرور در این حالت فوراً جواب «شماره پیدا نشد» بدهد، از روی مدت پاسخ می‌توان فهمید کدام شماره‌ها ثبت‌اند و کدام نه (وقتی شماره ثبت باشد، سرور وقت بیشتری صرف مقایسهٔ گذرواژه می‌کند). قطعه کد زیر این نشت را می‌بندد و رفتارش آزمون خودکار دارد.

در نمودار زیر – و همهٔ نمودارهای «جریان» در این فصل – هر ستون عمودی یک بازیگر است (کاربر، مرورگر، یک مسیر، پایگاه داده)، پیکان‌ها پیام‌هایی‌اند که در طول زمان از بالا به پایین رد و بدل می‌شوند، و کادر `alt / else` یعنی «بسته به شرط، یکی از این دو شاخه اجرا می‌شود».

```
export async function verifyCustomerCredentials(phone, password) {
  const user = await prisma.user.findUnique({ where: { phone } });
  // با یک هش ثابت مقایسه می‌کند passwordMatches، اگر کاربری نباشد:
  // هزینه زمانی «کاربر نیست» و «گذرواژه اشتباه است» یکی است
  if (!(await passwordMatches(password, user?.passwordHash))) return null;
  return { id: user.id, name: user.name, phone: user.phone, sessionVersion: user.sessionVersion };
}
```

قطعه کد ۵-۲ – بستن نشت زمانی: در حالت «کاربر وجود ندارد» هم یک هش الکی روی یک مقدار ثابت گرفته می‌شود، تا مدت پاسخ نگوید کدام شماره در سامانه ثبت است.



شکل ۵-۲ – جریان ورود. پیام خطا در هر دو حالت «شماره ناموجود» و «گذرواژه اشتباه» یکسان است تا شمارش کاربران ممکن نشود.

نشستی که می‌گوید مال کیست. مشتری‌ها و کارکنان در دو جدول جدا با شناسه‌های هم‌پوشان نگه داشته می‌شوند – جدایی حساب کارکنان از حساب مشتری الگوی رایج محصولات تجاری است. در نسخهٔ نخست،

توکن فقط یک «شناسه» و یک «نقش» داشت و هر فراخواننده باید از روی نقش حدس می‌زد شناسه مال کدام جدول است؛ یک مسیر همین را فراموش کرده بود و آگهی ثبت‌شده با نشست کارمند را به مشتری‌ای نسبت می‌داد که شناسه عددی‌اش یکی بود. اکنون توکن نوع حساب (customer یا staff) را با خودش دارد و هنگام خواندن با Zod سنجیده می‌شود، و هر مسیر با access اعلام می‌کند کدام نوع را می‌پذیرد؛ نوع «بازیگر»ی که به منطق مسیر می‌رسد با همان اعلام یکی است، پس این خطا دیگر کامپایل نمی‌شود. توکن کارمند در هر درخواست با ردیف خودش در پایگاه داده تطبیق داده می‌شود تا حساب غیرفعال‌شده همان لحظه دسترسی‌اش را از دست بدهد.

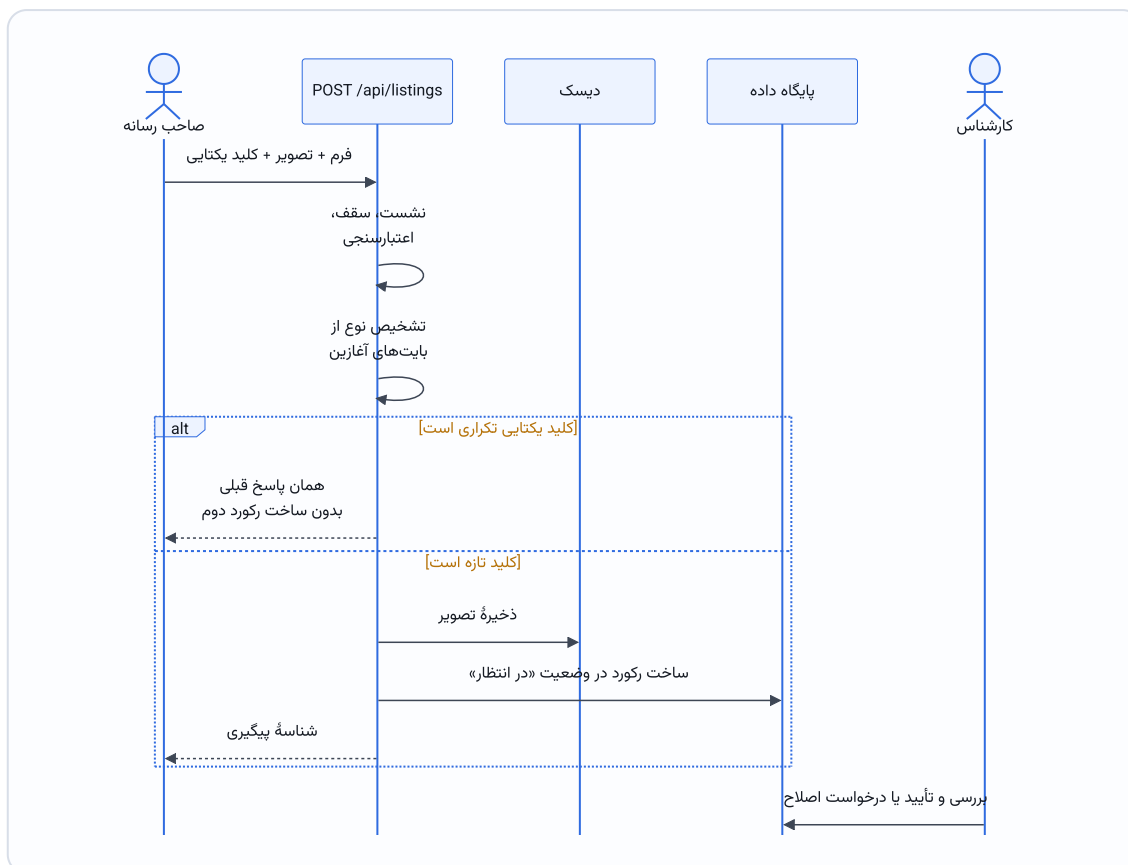
دسترسی مدیران چهار سطح پلکانی دارد و در هر مسیر مدیریتی، سطح لازم صریحاً بررسی می‌شود. نکته کلیدی آنکه این بررسی روی **نقطه پایانی** انجام می‌شود، نه در رابط کاربری: پنهان کردن یک دکمه امنیت نیست، چون درخواست را می‌توان مستقیم ساخت.

۳-۵ ثبت آگهی و بارگذاری امن فایل

صاحب رسانه فرمی چندمرحله‌ای پر می‌کند و تصویر بارگذاری می‌کند. پذیرش فایل از عموم خطرناک‌ترین نقطه ورودی هر سامانه است، پس نوع فایل از **بایت‌های آغازین خود فایل** تشخیص داده می‌شود، نه از نوع اعلام‌شده مرورگر. فایلی که ادعا می‌کند تصویر است ولی امضای بایتی تصویر ندارد، رد می‌شود.

```
function sniff(buf: Buffer): ImageKind | null {
  if (buf.length >= 3 &&
      buf[0] === 0xff && buf[1] === 0xd8 && buf[2] === 0xff) return "jpeg";
  if (buf.length >= 8 &&
      buf[0] === 0x89 && buf[1] === 0x50 && buf[2] === 0x4e && buf[3] === 0x47 &&
      buf[4] === 0x0d && buf[5] === 0x0a && buf[6] === 0x1a && buf[7] === 0x0a)
    return "png";
  if (buf.length >= 12 &&
      buf.toString("ascii", 0, 4) === "RIFF" &&
      buf.toString("ascii", 8, 12) === "WEBP") return "webp";
  return null;
  // ادعای مرورگر خوانده نمی‌شود
}
```

قطعه کد ۳-۵ – تشخیص نوع تصویر از بایت‌های آغازین خود فایل. نوع اعلام‌شده مرورگر اصلاً مبنا قرار نمی‌گیرد.



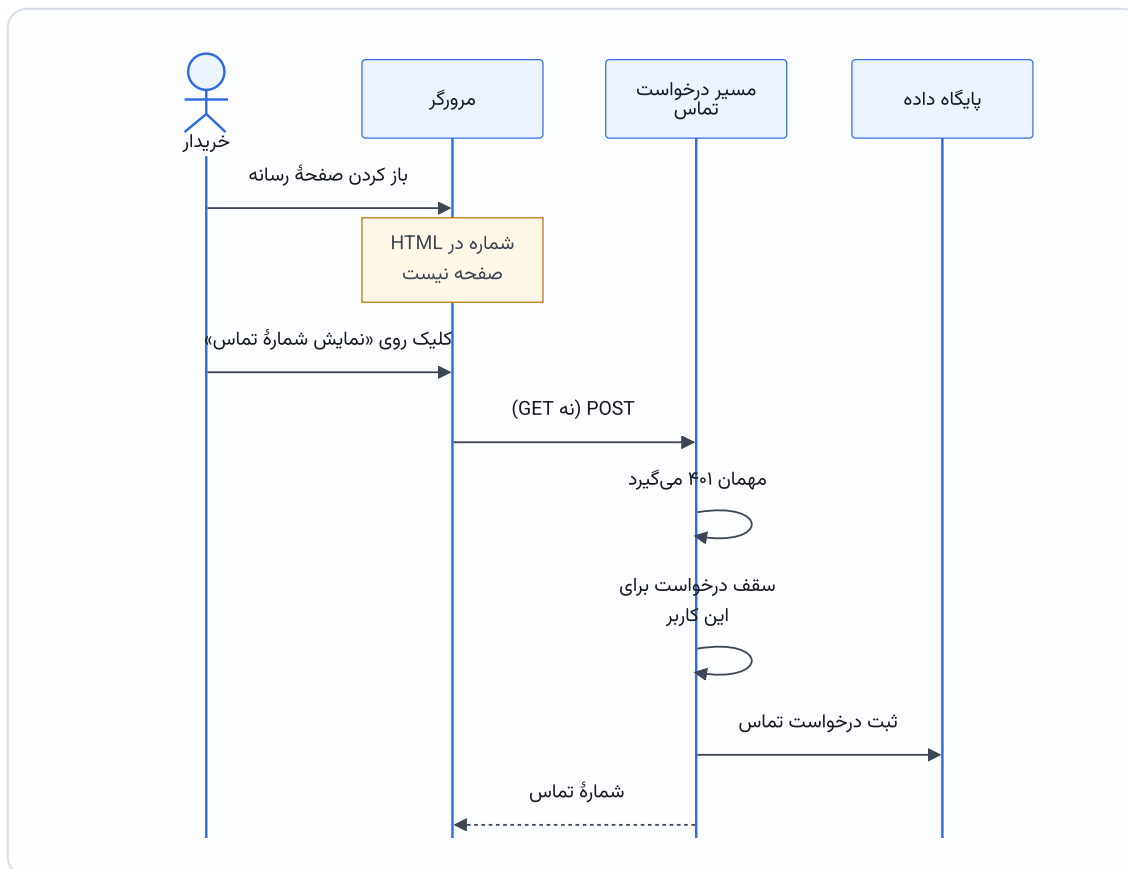
شکل ۵-۳ - ثبت آگهی. کلید یکتایی تضمین می‌کند ارسال دوبارهٔ فرم، آگهی دوم نسازد.

فرم چندمرحله‌ای ثبت آگهی

شکل ۵-۴ - فرم ثبت آگهی از دید صاحب رسانه. هر مرحله پیش از رفتن به بعدی اعتبارسنجی می‌شود و دکمهٔ ارسال هنگام پرواز درخواست غیرفعال می‌ماند.

۴-۵ محافظت از دادهٔ تماس مالکان

ارزشمندترین دادهٔ این سامانه فهرست شماره‌های تماس مالکان است و ساده‌ترین راه نابودی ارزش آن، قراردادنش در HTML صفحهٔ عمومی است. در این پیاده‌سازی شماره **هرگز** در صفحه قرار نمی‌گیرد (حتی برای کاربر واردشده) و تنها در پاسخ یک درخواست جداگانه ارسال می‌شود.



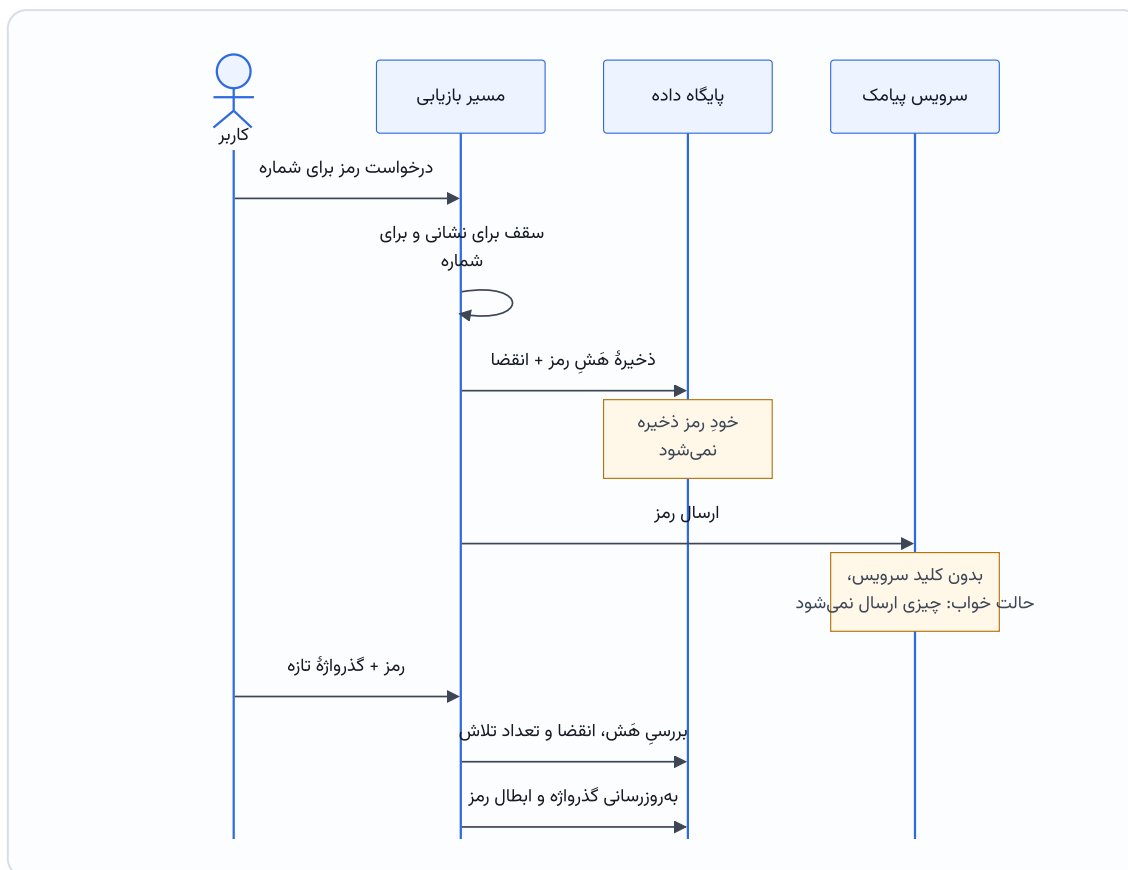
شکل ۵-۵ - دریافت شمارهٔ تماس. سه لایه (نیود شماره در صفحه، الزام ورود، و سقف درخواست) برداشت انبوه را پرهزینه می‌کنند.

ادعای صادقانه

این سازوکار برداشت انبوه را ناممکن نمی‌کند؛ آن را پرهزینه می‌کند. مهاجمی که حاضر باشد حساب بسازد، وارد شود و سقف درخواست را رعایت کند، در نهایت می‌تواند داده را جمع کند. هدف طراحی، بالا بردن هزینه تا جایی است که برداشت انبوه صرف نکند - و ادعای بیش از این، ادعای نادرستی است.

۵-۵ بازیابی گذرواژه با رمز یک‌بارمصرف

بازیابی گذرواژه در سه گام انجام می‌شود: درخواست رمز، تأیید رمز، و تعیین گذرواژهٔ تازه. خود رمز هیچ‌جا ذخیره نمی‌شود؛ فقط هَش آن - همان اثر یک‌طرفه‌ای که پیش‌تر توضیح داده شد - به‌همراه زمان انقضا در پایگاه داده می‌ماند.



شکل ۵-۶ - بازیابی گذرواژه. لایه پیامک بدون کلید سرویس در حالت خواب می‌ماند؛ کد کامل است ولی پیامی ارسال نمی‌شود.

۵-۶ همروندی و صحت زیر بار همزمان

سه نوع خطای همروندی در این سامانه به‌صورت ساختاری بسته شده‌اند.

ارسال دوباره یک عملیات. کاربری که دکمه ثبت را دو بار می‌زند یا اتصالش قطع می‌شود و دوباره می‌فرستد، نباید دو آگهی بسازد. هر ثبت آگهی یک کلید یکتایی همراه دارد؛ اگر همان کلید دوباره برسد، پاسخ ذخیره‌شده بازگردانده می‌شود و رکورد دومی ساخته نمی‌شود.

تصمیم دوباره روی یک آگهی. اگر دو کارشناس همزمان یک آگهی را تأیید کنند، پلن ویژه نباید دو بار اعطا شود. تصمیم تأیید تک‌شلیک است: تلاش دوم پاسخ «تعارض» می‌گیرد. نکته فنی این تضمین آن است که شرط «هنوز بررسی نشده» در خود دستور نوشتن نشسته است، نه در یک خواندن جداگانه پیش از آن؛ وگرنه دو درخواست همزمان هر دو می‌توانستند از شرط رد شوند و هر دو بنویسند. مصرف کد یک‌بارمصرف بازیابی گذرواژه نیز از همین الگو استفاده می‌کند: کد با همان دستوری مصرف‌شده علامت می‌خورد که آن را می‌خواند، پس یک کد دو بار جواب نمی‌دهد.

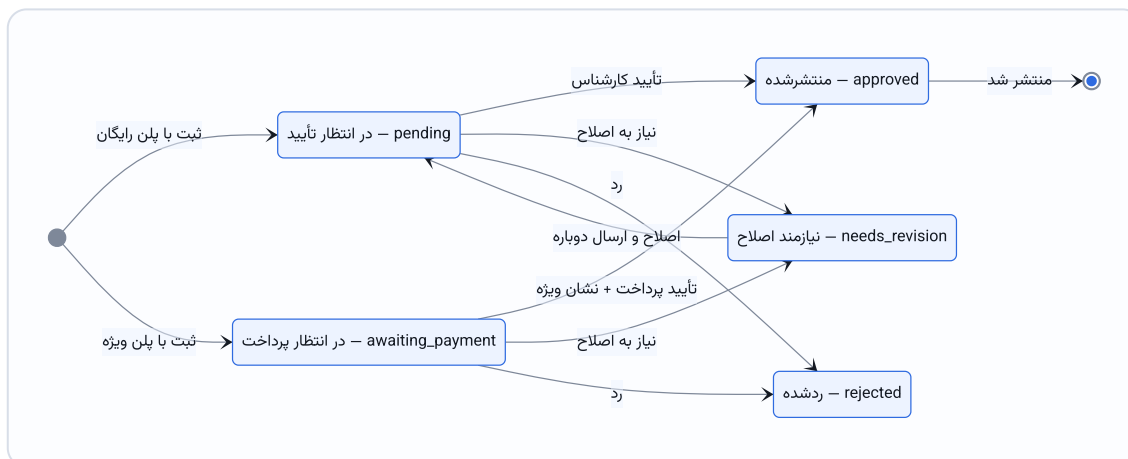
خواندن، تغییر، نوشتن. ثبت یک نظر، میانگین امتیاز رسانه را هم تغییر می‌دهد. اگر این دو جدا انجام شوند، دو نظر همزمان می‌توانند میانگین را خراب کنند. هر دو در یک تراکنش انجام می‌شوند: یا هر دو یا هیچ‌کدام.

```
// شرط «هنوز منتشر نشده» داخل خود دستور نوشتن است، نه در یک خواندن
// جداگانه پیش از آن: از دو تصمیم همزمان فقط یکی میتواند ردیف را جابه‌جا کند
// نسخه‌ای است که کارشناس دید؛ اگر فرستنده در این فاصله محتوا را عوض کرده seen
// باشد، شرط برقرار نیست و محتوای بررسی‌نشده منتشر نمی‌شود
const { count } = await prisma.billboard.updateMany({
  where: { id, moderation: { in: UNDECIDED }, plan: before.plan, updatedAt: seen },
  data: { ...decisionOutcome(decision, before.plan), reviewNote: note },
});
// یعنی نگهبان ردش کرد؛ خط لوله این خطا را به ۴۰۹ ترجمه می‌کند
if (count === 0) throw conflict("گهی قبلاً بررسی شده یا پس از باز کردن شما تغییر کرده است")
```

قطعه کد ۴-۵ – تصمیم تک‌شلیک. شرط و نوشتن یک عملیات اتمی‌اند، پس تلاش دوم برای همان گذار پاسخ «تعارض» می‌گیرد – حتی اگر دقیقاً همزمان برسد – و تأیید فقط روی همان نسخه‌ای می‌نشیند که کارشناس دیده است. همین الگو برای ارسال دوباره آگهی و برای مصرف کد یک‌بارمصرف نیز به کار رفته است. خود قاعده گذار (decisionOutcome) یک تابع خالص در lib/domain است و آزمون واحد دارد.

در فرم ثبت آگهی، صاحب رسانه میان دو پلن یکی را برمی‌گزیند: **رایگان** یا **ویژه**. آگهی رایگان پس از تأیید کارشناس منتشر می‌شود. آگهی ویژه همان بررسی را دارد به‌اضافه یک پرداخت که – چون سامانه درگاه بانکی ندارد (بخش ۴-۳) – کارشناس آن را دستی تأیید می‌کند؛ در افزایش، آن آگهی نشان «ویژه» می‌گیرد و در فهرست بالاتر دیده می‌شود.

وضعیت بررسی هر آگهی – ستون moderation، جدا از آزاد یا مشغول بودن سازه – با یک state machine مدل شده است: چند وضعیت مشخص، و فهرستی از گذارهای مجاز میان آنها. آگهی رایگان از وضعیت pending (در انتظار تأیید) آغاز می‌شود و آگهی ویژه از awaiting_payment (در انتظار پرداخت). هر گذار فقط از یک وضعیت معین ممکن است، پس «کلیک دوم» روی یک تصمیم – که دیگر از وضعیت درست شروع نمی‌شود – رد می‌شود.



شکل ۷-۵ – ماشین حالت یک آگهی. نام انگلیسی هر وضعیت همان است که در پایگاه داده ذخیره می‌شود. چون هر پیکان تنها از یک وضعیت آغاز می‌شود، دو کارشناس نمی‌توانند یک آگهی را دو بار تأیید کنند: تلاشی دوم پاسخ «تعارض» (کد ۴۰۹) می‌گیرد.



صف بررسی آگهی‌ها در پنل مدیریت

شکل ۸-۵ – همان ماشین حالت در عمل: نشان‌های «در انتظار تأیید»، «رد شده» و «در انتظار پرداخت» کنار سه تصمیم مجاز کارشناس. توضیح برای فرستنده در حالت «نیاز به اصلاح» و «رد» الزامی است.

۷-۵ مدل تخمین بازدید روزانه

منابع داده موجود برای هر سازه یک عکس، یک اندازه و یک نام خیابان منتشر می‌کنند و نه بیشتر. اما عددی که خریدار واقعاً برای تصمیم به آن نیاز دارد – «چند نفر در روز این سازه را می‌بینند» – در هیچ منبعی نیست. مدلی زیر آن را از همان داده در دسترس برآورد می‌کند و از همان سه‌گامی پیروی می‌کند که صنعت جهانی تبلیغات محیطی به کار می‌برد: نخست عبور روزانه از کنار سازه شمرده می‌شود، بعد به «نفر» تبدیل می‌شود، و در پایان با ضربی کوچک‌تر از یک ضرب می‌شود که می‌گوید چه سهمی از این جمعیت واقعاً امکان دیدن این سازه مشخص را دارد.

```
vehicles = roadClassbase × cityFactor × spread
opportunity = vehicles × occupancy + pedestrians
visibility = seeroad × seetype × (1 + sizeBonus)
impressions = opportunity × visibility
```

راهنمای نمادها – $roadClass_{base}$: عبور پایه آن نوع معبر (جدول زیر) · $cityFactor$: ضریب اندازه شهر · $spread$: تنوع کوچکی که از شناسه خود رکورد می‌آید تا دو سازه در یک خیابان عدد یکسان نگیرند · $occupancy$: سرنشین به‌ازای هر خودرو (۱/۵۵) · $pedestrians$: عابران پیاده · see_{road} ، see_{type} : چه سهمی از عبور آن معبر و آن نوع رسانه فرصت خواندن پیام را دارد · $sizeBonus$: افزایش ناشی از بزرگی سازه (حداکثر ۳۰٪) · $impressions$: برآورد نهایی بازدید روزانه.

۷-۱-۵ کلاس معبر، نه جمعیت شهر

پایه مدل، **کلاسِ معبر** است (بزرگراه، بلوار، تقاطع، ...) و نه جمعیتِ شهر. این تمایز، اصلاح یک خطای واقعی در نسخه نخستِ مدل بود: آن نسخه کارِ هر سازه تهران را با درصدی از جمعیتِ نه‌میلیونی شهر آغاز می‌کرد و در نتیجه یک تابلو در یک کوچه فرعی هم ادعای ۴۵۰,۰۰۰ بیننده در روز داشت. حقیقت این است که یک بزرگراه در هر شهری ترافیکِ بزرگراه دارد؛ جمعیتِ شهر فقط اندکی بزرگ‌ترش می‌کند – نه به همان نسبت.

جدول ۱-۵ – کلاس‌های معبر و ضرایب هر یک. ستون «سهم دیدن» نسبت جمعیتی است که امکان واقعی خواندن پیام را دارد.

کلاس معبر	عبور روزانه	ضریب پیاده	سهم دیدن
بزرگراه و آزادراه	۱۲۰,۰۰۰	۰/۰۱	۳۴٪
بلوار و خیابان اصلی	۵۲,۰۰۰	۰/۰۹	۴۲٪
تقاطع و میدان	۳۸,۰۰۰	۰/۲۲	۵۵٪
خیابان معمولی	۱۷,۰۰۰	۰/۱۸	۴۶٪
ایستگاه حمل‌ونقل	۶,۰۰۰	۲/۴۰	۶۱٪
معبر محلی	۷,۰۰۰	۰/۱۴	۴۰٪

کلاس هر سازه با تطبیق واژگان فارسی نشانی آن تعیین می‌شود و تطبیق از بلندترین کلیدواژه آغاز می‌شود تا نام یک مسیر مشخص بر واژه عام درون آن اولویت بگیرد. هر نشانی ناشناخته به «خیابان معمولی» نگاشت می‌شود – که انتخاب صادقانه‌ای است، چون بیشتر نشانی‌های موجود در داده همین‌اند.

۵-۷-۲ سه ملاحظهٔ روش‌شناختی

اثر جمعیتِ شهر خطی نیست. ضریبِ شهر برابر است با «نسبتِ جمعیت به یک میلیون» به توان $۰/۳۵$ – یعنی یک توانِ کسری، نه ضربِ ساده. تهران ۲۳ برابر زنجان جمعیت دارد، ولی خیابان‌هایش ۲۳ برابر خودرو جابه‌جا نمی‌کنند؛ حدودِ سه برابر. این توانِ کسری روشِ استانداردِ بیانِ همین نکته است و پراکندگیِ عددها را در کل فهرست از حدودِ ۷۰ برابر به حدودِ ۴ برابر می‌آورد.

اندازه بازده نزولی دارد. دو برابر کردن سطح یک سازه، تعداد خوانندگانش را دو برابر نمی‌کند؛ بنابراین مساحت با جذر وارد فرمول می‌شود و اثرش به حداکثر ۳۰٪ افزایش محدود است. نسبت نهایی دیده‌شدن نیز میان ۱۸٪ و ۸۲٪ مقید می‌ماند.

خروجی قطعی است. مدل هیچ عددِ تصادفی‌ای ندارد. تنوع لازم (تا دو سازه در یک خیابان عددِ یکسان نگیرند) از هَش‌کردنِ شناسهٔ خودِ رکورد به دست می‌آید؛ چون شناسه ثابت است، همان رکورد در هر اجرا دقیقاً همان عدد را می‌گیرد. نسخهٔ نخست از یک ضریبِ تصادفی استفاده می‌کرد و نتیجه‌اش این بود که هیچ عددی در گزارش بازتولیدپذیر نبود.

مرز اعتبار مدل

این یک **برآورد** است، نه اندازه‌گیری، و سامانه هم همین را به کاربر می‌گوید. اعتبارش از دو چیز می‌آید: پیروی از شکلی که صنعت به کار می‌برد، و ثبت منبع هر ضریب در کد – جمعیتِ شهرها از سرشماری، ضریبِ ۱/۵۵ نفر بر خودرو از ارقام منتشرشدهٔ شرکتِ کنترلِ ترافیکِ تهران برای سفرهای شهری، و قیمت‌های مرجع مدل از میانهٔ قیمت‌هایی که خودِ منابع منتشر کرده‌اند. آنچه این مدل ادعا نمی‌کند، جایگزینی یک سرشماری میدانی است.

۸-۵ مدیریت خطا و ثبت رخداد

قاعده حاکم این است: کاربر هرگز نباید صفحه سفید یا پیام فنی خام ببیند، و توسعه‌دهنده هرگز نباید از فهمیدن اینکه چه چیزی خراب شده ناتوان بماند. این دو خواسته با هم در تعارض‌اند و پاسخ، جدا کردن این دو مخاطب از هم است.

هنگام بروز یک خطای پیش‌بینی‌نشده، یک شناسه کوتاه ساخته می‌شود. کاربر یک پیام آرام فارسی به‌همراه همان شناسه می‌بیند؛ اما stack trace (فهرست توابعی که تا لحظه خطا صدا زده شده بودند)، خود پرس‌وجو و جزئیات درونی فقط در گزارش سرور و کنار همان شناسه نوشته می‌شوند. کاربر شناسه را گزارش می‌کند و توسعه‌دهنده دقیقاً همان رخداد را پیدا می‌کند، بی‌آنکه چیزی از درون سامانه بیرون درز کرده باشد.

خطاهایی که پیش‌بینی‌شده‌اند – رسانه‌ای که نیست، آگهی‌ای که قبلاً بررسی شده – اصلاً «خطای سرور» نیستند. لایه داده آن‌ها را به شکل یک `DomainError` با نوع (یافت‌نشده، تعارض، نامعتبر...) و پیام فارسی اعلام می‌کند و خط لوله تنها جایی است که نوع را به کد وضعیت HTTP تبدیل می‌کند؛ پس یک «یافت‌نشده» نمی‌تواند در یک مسیر ۴۰۴ و در مسیر کناری ۴۰۰ باشد. پیام‌های پیش‌فرض اعتبارسنجی هم فارسی شده‌اند تا طرح‌واره‌ای که پیام خودش را ندارد، به کاربر فارسی‌زبان انگلیسی جواب ندهد.

هر رخداد به‌صورت یک شیء JSON در یک خط ثبت می‌شود. این قالب برای چشم انسان بهینه نیست، ولی برای پردازش ماشینی آماده است: اگر بعدها یک انباشتگر رخداد اضافه شود، هیچ تغییری در کد لازم نخواهد بود.

این تضمین هر دو مسیر داده فصل چهارم را می‌پوشاند، نه فقط مسیر API را. صفحه‌ای که خودش مستقیم از پایگاه داده می‌خواند، بیرون از چرخه عمر یک مسیر HTTP رندر می‌شود؛ اگر رندر شکست بخورد، چارچوب شناسه خودش را به کاربر نشان می‌دهد و اگر کاری نکنیم، آن شناسه هرگز به گزارش سرور نمی‌رسد – یعنی دقیقاً همان شناسه‌ای که کاربر گزارش می‌کند، در فایل گزارش پیدا نمی‌شود. به همین دلیل خطاهای رندر سمت سرور از راه قالب `onRequestError` در `instrumentation.ts` به همان گزارش ساختاریافته هدایت می‌شوند و همان شناسه در هر دو سو یکی است.

افزون بر این، هر تغییر مدیریتی (تأیید، رد، حذف، تغییر نقش) در یک رد ممیزی ماندگار با نشانی شبکه و جزئیات تغییر ثبت می‌شود؛ انجام‌دهنده با کلید خارجی واقعی به حساب خودش – کارمند یا مشتری – وصل است.

ارزیابی و آزمون

۱-۶ روش و نتایج آزمون خودکار

ارزیابی بر سه پایه انجام شده است: آزمون خودکار، یک بازبینی بدبینانه معماری، و یک خودارزیابی صریح که نقاط ضعف را هم فهرست می‌کند. آزمون‌ها در سه لایه‌اند. ۹ آزمون واحد قاعده‌های خالص محصول را (گذارهای وضعیت آگهی، قیمت‌های مشتق، شکل ستون‌های JSON) بدون ساخت و بدون پایگاه داده، در کمتر از یک ثانیه می‌سنجند؛ این ممکن است چون `lib/domain/` به حکم قاعده ESLint هیچ ورودی/خروجی‌ای ندارد. بقیه آزمون‌ها «در سطح واسط برنامه‌نویسی»اند؛ یعنی مثل یک کاربر بیرونی به آدرس‌های `/api/` درخواست واقعی HTTP می‌فرستند و پاسخ را می‌سنجند — نه اینکه توابع داخلی را جدا جدا صدا بزنند. آزمون‌ها روی یک نسخه ساخته‌شده (تولیدی) اجرا می‌شوند و نه سرور حالت توسعه: سرور حالت توسعه هر درخواست را در همان لحظه از نو ترجمه می‌کند و وقتی یکی از آزمون‌ها فهرست را صداها بار پیایی می‌خواند، از مهلت پاسخ می‌گذرد و کل اجرا گیر می‌کند. هر اجرا هم پایگاه داده جداگانه خودش را از نو می‌سازد تا نتیجه به وضعیت اجرای پیشین وابسته نباشد.

۱~ دقیقه

زمان اجرا، با ساخت

۱۰

جریان مرورگر، همه موفق

۱۷۹

موفق

۱۷۹

آزمون (۱۲ واحد، ۷۱۶۰ API، واردکننده)

هیچ‌یک از آزمون‌ها به سرویس بیرونی وابسته نیست؛ لایه پیامک هم بدون کلید سرویس خاموش می‌ماند، پس آزمون‌های رمز یک‌بارمصرف، کد را داخل همان پایگاه داده محلی می‌خوانند و هیچ پیامی فرستاده نمی‌شود.

جدول ۱-۶ — سرفصل‌های پوشش آزمون خودکار

سرفصل	آنچه سنجیده می‌شود
اعتبارسنجی ورودی	رد ورودی بدشکل، بیش‌ازحد بزرگ، یا خارج از فهرست مجاز برای مرتب‌سازی و پالایش
کنترل دسترسی	دسترسی سطح‌به‌سطح مدیران و دسترسی در سطح شیء — کاربر به داده کاربر دیگر نرسد
سقف نرخ درخواست	فعال‌شدن سقف و بازگشت پاسخ درست پس از پایان بازه
عدم شمارش کاربران	یکسان بودن پاسخ برای شماره ناموجود و گذرواژه اشتباه — هم در متن پاسخ و هم در زمان پاسخ
بارگذاری فایل	رد فایلی که ادعای تصویر دارد ولی امضای بایتی تصویر ندارد
ماشین حالت تأیید	مجاز بودن فقط گذارهای تعریف‌شده و تک‌شلیک بودن هر تصمیم
بازیابی گذرواژه	چرخه کامل رمز یک‌بارمصرف، انقضا و محدودیت تعداد تلاش

سرفصل	آنچه سنجیده می‌شود
درستی مرتب‌سازی	درست بودن ترتیب خروجی – با یک محافظ که مانع می‌شود آزمون روی فهرست خالی «به‌دروغ» سبز شود
رفتار وابسته به نوع اتصال	روشن‌شدن نشانه «امن» روی کوکی وقتی سایت پشت یک پروکسی HTTPS است، و روشن‌نشدنش روی اتصال ساده محلی
واردکننده شبانه	اینکه به‌روزرسانی خودکار داده هرگز تصحیح دستی مدیر را باطل نکند، سازه‌ای را که مدیر از سایت برداشته برنگرداند، و ردیفی که عمداً حذف شده دوباره نسازد
قاعده‌های دامنه (آزمون واحد)	گذارهای ماشین حالت، پلن ویژه فقط با تأیید، قیمت‌های مشتق، و شکل ستون‌های JSON
پنل مدیریت	هر بخش نشانی خودش را دارد، نشانی‌های قدیمی هدایت می‌شوند، و حساب غیرفعال‌شده به صفحه ورود فرستاده می‌شود

آزمون‌های بالا سامانه را از راه HTTP می‌سنجند و چیزی را که فقط در مرورگر رخ می‌دهد نمی‌بینند. برای همین ۱۰ جریان کاربری با `npm run test:e2e` در کروم نصب‌شده و از راه پروتکل کنترل آن اجرا می‌شوند – بدون افزودن هیچ وابستگی تازه‌ای به پروژه: دیدن فهرست و باریک‌شدنش با پالایه، ورود و رد ورود نادرست، ثبت‌نام دومرحله‌ای، آشکار شدن شماره تماس پس از ورود، ثبت آگهی و دیدنش در حالت «در انتظار»، رد ورود کاربر عادی به پنل مدیریت، ورود کارشناس و جابه‌جایی میان بخش‌های پنل از راه نشانی هر بخش، و کارکردن فهرست در عرض تلفن همراه. این مجموعه سه اشکال واقعی را پیدا کرد که آزمون سطح HTTP نمی‌توانست ببیند.

آزمون‌های محافظ

چند آزمون، رفتار کاربر را نمی‌سنجند بلکه قواعد پروژه را می‌سنجند: اینکه هر مسیر از خط لوله بگذرد، فقط مسیرهای خروج بتوانند بی‌سقف نرخ باشند، هر پاسخ «تعداد زیاد» از یک تابع مشترک عبور کند و هیچ انیمیشن بی‌پایانی بدون توقف در زبانه پنهان نماند. کنار آن‌ها، قاعده‌های ESLint مرزهای معماری را نگه می‌دارند: اتصال پایگاه داده فقط در `lib/db/`، مجموعه داده اولیه فقط در اسکریپت پرکردن، و هیچ ورودی/خروجی در `lib/domain/`. این‌ها جلوی بازگشت خطاهای گذشته را می‌گیرند.

۲-۶ یک کلاس خطا و درسی که از آن گرفته شد

در جریان توسعه، شش خطای جداگانه پیدا شد که همه یک ریشه داشتند: **کد فرض کرده بود ویژگی‌های اتصال کاربر همان است که روی رایانه توسعه‌دهنده دیده می‌شود** – مثلاً «چون الان روی `localhost` هستیم، پس اتصال امن نیست». این خطاها روی رایانه توسعه‌دهنده کاملاً نامرئی بودند و فقط وقتی بیرون می‌زدند که سایت از دستگاه دیگری روی شبکه محلی باز شود – دقیقاً همان کاری که داور با تلفن همراهش می‌کند.

جدول ۲-۶ – شش خطای هم‌ریشه و اصلاح هر یک

خطا	اصلاح
نشانه «امن» روی کوکی از یک متغیر محیطی زمان ساخت خوانده می‌شد، نه از خود اتصال	خوانده‌شدن از پروتکل واقعی همان درخواست

خطا	اصلاح
بررسی مبدأ، آدرس را با نامی که سرور روی آن بالا آمده مقایسه می‌کرد	مقایسه با سرآیند «میزبان» خود درخواست
نسخه‌برداری متن با واسط استاندارد مرورگر، بیرون از یک بستر امن کار نمی‌کرد	یک تابع مشترک با مسیر جایگزین برای این حالت
ساخت نشانی کامل سایت به localhost برمی‌گشت و لینک‌ها روی موبایل می‌شکست	یک منبع واحد برای نشانی سایت در کل پروژه
ویژگی lazy load (بارگذاری تصویر با تأخیر تا لحظه دیده‌شدن) روی کارت‌هایی فعال بود که در یک نوار چرخان‌اند و هیچ‌گاه وارد دید نمی‌شوند	حذف آن از عناصری که کاربر نمی‌تواند به آن‌ها اسکرول کند
انیمیشن بی‌پایان در زبانه پنهان هم ادامه می‌یافت و پردازنده را بیدار نگه می‌داشت	افزودن به فهرست توقف هنگام پنهان‌شدن صفحه

درس این تجربه فراتر از شش اصلاح است. پس از شناسایی الگو، یک **قاعده دائمی** در سند قواعد پروژه ثبت شد به‌همراه یک پرسش آزمون: «آیا این تغییر وقتی سایت از دستگاه دیگری روی شبکه محلی باز شود هم کار می‌کند؟» دو مورد از این رفتارها اکنون آزمون خودکار دارند.

۳-۶ بازبینی بدبینانه معماری

بازبینی‌های پیشین این کد، نخست مستندات را می‌خواندند و سپس کد را از دریچه آن. در یک بازبینی آخر، فرض برعکس شد: ساختار بی‌طراحی و به‌مرور رشد کرده است، و باید ایراد ریشه‌ای را پیدا کرد نه ایراد کوچک را. نتیجه نه یافته بود. هشت مورد اصلاح شد و برای هر یک ابزاری – نه یک پاراگراف – نگه‌بان شد؛ جدول زیر آن‌ها را خلاصه می‌کند.

جدول ۳-۶ – یافته‌های بازبینی بدبینانه و آنچه جایشان نشست

یافته	جایگزین	نگهبان
ترتیب امنیتی بررسی‌ها در ۳۵ مسیر دستی تکرار شده بود	خط لوله واحد <code>defineRoute</code>	نوع TypeScript و آزمون نگهبان
۲۴ مسیر از ۳۵ مستقیم پایگاه داده را صدا می‌زدند؛ منطق کسب‌وکار داخل مسیرها بود، گاهی دو بار	لایه دسترسی داده <code>server-only</code> و قاعده‌های خالص در <code>lib/domain/</code> – الگوی پیشنهادی مستندات Next.js	ESLint؛ ۳۰ فایل واردکننده اتصال < ۱۳
نشست نمی‌گفت مال کدام نوع حساب است؛ یک مسیر شناسه کارمند را در کلید مشتری نوشت	توکن نوع‌دار و «بازیگر» نوع‌دار؛ کلیدهای خارجی واقعی برای پاسخ و ممیزی	نوع TypeScript
یک ستون <code>status</code> به دو پرسش جواب می‌داد	دو ستون شماری، و جدول جدا برای دفتر خزنده	قیدهای نوع شماری و آزمون
ستون‌های JSON بی‌بررسی باور می‌شدند	خواندن از راه طرح‌واره Zod	آزمون واحد

یافته	جایگزین	نگهبان
صفحه‌های پشتِ ورود، صفحهٔ اولشان را در مرورگر می‌گرفتند؛ پنل یک مؤلفهٔ ۴۰۰ خطی با هشت زبانه بود	صفحهٔ اول با داده روی سرور؛ هر بخشِ پنل یک نشانی، با بررسیِ حساب روی سرور	آزمون API و جریانِ مرورگر
انبارۀ مشترکِ خاموش نیمه‌کاره بود؛ کش مشترک می‌شد، شمارندۀ سقفِ نرخ نه	یک الگوریتم، در حافظه و به‌صورتِ اسکرپتِ اتمی در Redis، با بازگشتِ فوری به حافظه	مجموعهٔ آزمون با Redis روشن، خاموش و بدون آن
قواعدِ مهم فقط در مستندات بودند و برخی توضیح‌ها کهنه شده بودند	قاعده‌ها به ابزار منتقل شدند	—

تغییرِ طرح‌وارهٔ پایگاه داده پیش از اجرا روی رونوشتِ پایگاه دادهٔ واقعی آزموده شد: ۳,۵۳۶ ردیف پیش و پس، بررسیِ یکپارچگی و کلیدهای خارجی پاک، و هیچ اختلافی میانِ پایگاه دادهٔ مهاجرت‌یافته و طرح‌واره. در همین مسیر چند خطای رفتاریِ واقعی هم پیدا و با آزمون بسته شد؛ از جمله ذخیرهٔ هر نشانیِ دلخواه به‌عنوانِ عکسِ یک رسانه از پنل مدیریت، و شمارش‌های آماری که ردیف‌های ردشده را هم می‌شمردند.

۴-۶ خودارزیابی

جدول زیر ارزیابیِ نگارنده از محورهای مختلفِ کار است، همراه با نقطهٔ ضعفِ هر محور.

جدول ۴-۶ – خودارزیابیِ محورهای پروژه

محور	شواهد	نقطهٔ ضعف
معماری	یک Data Access Layer که تنها در پایگاه داده است، دو مسیر دادهٔ چارچوب هرکدام سر جای درستش، یک خط لوله برای هر ۳۵ مسیر، و مرزهایی که ESLint نگه می‌دارد	یک بار، کل مجموعهٔ داده در بسته‌ای که به مرورگر می‌رفت جا خوش کرده بود؛ خطا با اندازه‌گیری پیدا و به‌صورتِ ساختاری اصلاح شد
پایگاه داده	طرح‌واره‌ای که درست normalize شده، نمایه‌های ترکیبیِ منطبق بر پرس‌وجوهای واقعی، مهاجرتِ نسخه‌بندی‌شده	موتور تک‌فایلی است، و چند فیلد (مثل فهرستِ تصویرها) به‌جای جدولِ جداگانه، به‌صورتِ JSON درون یک ستون ذخیره شده‌اند – یک سازشِ آگاهانه که اکنون با اعتبارسنجی هنگام خواندن جبران می‌شود
امنیت	مرز دسترسی روی خودِ نقطهٔ پایانی، چهار سطح دسترسی، کوکی امضاشده، سقف نرخ، رد ممیزی، پیام خطای عمومی	بدون فایروالِ کاربردی (WAF) و بدون آزمونِ نفوذ بیرونی
همروندی	کلیدِ یکتا روی ثبتِ آگهی، تصمیمِ تأییدِ تک‌شلیک، تراکنش روی بازمحاسبهٔ امتیاز	شمارنده‌های سقفِ نرخ با یک متغیرِ محیطی به Redis منتقل می‌شوند، ولی Redis مستقر نیست و رفتارِ چندنمونه‌ای روی سرورِ واقعی سنجیده نشده
آزمون	۱۷۹ آزمونِ خودکار (واحد، API و واردکننده) روی نسخهٔ ساخته‌شده و پایگاه دادهٔ جداگانه، و ۱۰ جریانِ مرورگر	آزمونِ تک‌مؤلفه‌ای وجود ندارد و همهٔ صفحه‌ها پوششِ مرورگری ندارند

محور	شواهد	نقطه ضعف
مدیریت خطا	صفحه‌های خطای فارسی، شناسه پیگیری، حالت‌های «خالی» و «در حال بارگذاری» طراحی شده	چند جریان هنوز نخستین نشانه خطا را به صورت کد وضعیت خام نشان می‌دهند
داده	۳,۵۳۶ رکورد از سه منبع، حذف تکراری، ۸۵٪ دارای مختصات	گردآوری یکباره نقطه‌ای است، نه جریانی که خودکار به روز شود؛ خزنده جزو سامانه در حال اجرا نیست

نتیجه‌گیری و کار آینده

۱-۷ دستاوردها

هدف این پایان‌نامه ساختن یک نمای واحد و قابل جست‌وجو از عرضه رسانه تبلیغاتی محیطی بود. سامانه‌ای ساخته شده که ۳,۵۳۶ رسانه از سه منبع را در یک فهرست یکپارچه گرد آورده، ۸۵٪ آن‌ها را روی نقشه نشانده، و برای همه‌شان عددی برای بازدید روزانه برآورد کرده که در هیچ منبع اصلی وجود نداشت.

- **یکپارچه‌سازی داده** – گردآوری از سه منبع، پاک‌سازی، حذف تکراری با تطبیق نام و موقعیت، و ژئوکد کردن ۳,۰۲۰ رکورد.
- **مدل تخمین بازدید** – قطعی، بازتولیدپذیر، با منبع ثبت‌شده برای هر ضریب؛ نقطه تمایز اصلی سامانه.
- **کیفیت مهندسی سمت سرور** – یک خط لوله واحد روی ۴۴ نقطه پایانی، یک لایه داده که تنها در پایگاه داده است، اعتبارسنجی طرح‌واره‌ای روی تمام ورودی‌ها، سه سازوکار مستقل برای صحت زیر بار هم‌زمان، و رد ممیزی ماندگار برای هر تغییر مدیریتی.
- **صحت سنجیده‌شده** – ۱۷۹ آزمون خودکار روی ساخت تولیدی و ۱۰ جریان مرورگر، همگی موفق.

۲-۷ محدودیت‌ها

فهرست زیر صادقانه است و برای پیش‌گیری از برداشت اغراق‌آمیز از نتایج آورده می‌شود:

1. **داده زنده نیست.** مجموعه فعلی یک گردآوری نقطه‌ای است. خزنه در مخزن وجود دارد ولی به‌صورت زمان‌بندی‌شده اجرا نمی‌شود، پس تغییر قیمت یا حذف یک سازه در منبع، خودکار بازتاب نمی‌یابد.
2. **تخمین بازدید، اندازه‌گیری نیست.** مدل از شکل صنعتی پیروی می‌کند ولی با شمارش میدانی اعتبارسنجی نشده است.
3. **یک نمونه اجرایی.** سامانه روی یک پردازنده مستقر است. کش و شمارنده‌های سقف نرخ با تنظیم یک متغیر محیطی میان چند نمونه مشترک می‌شوند، ولی آن انباره مستقر نیست. مسیر مهاجرت پایگاه داده به یک موتور کلاینت-سروری آزموده شده – ولی به‌عمد پیموده نشده، چون برای یک نمونه، فایل تک‌جا انتخاب بهتری است.
4. **آزمون مؤلفه وجود ندارد.** قاعده‌های دامنه آزمون واحد، واسط برنامه‌نویسی آزمون یکپارچه و جریان‌های اصلی آزمون مرورگر دارند؛ ولی تک‌تک مؤلفه‌های رابط کاربری دستی آزموده شده‌اند.
5. **پرداخت بیرون از سامانه است.** پلن ویژه با تأیید دستی کارشناس فعال می‌شود.
6. **کارایی زیر بار واقعی سنجیده نشده است.** رفتار سامانه هنگام درخواست هم‌زمان چند کاربر روی یک سرور واقعی آزموده نشده.

۳-۷ کار آینده

مسیر ادامه کار، به ترتیب اولویت و با در نظر گرفتن اینکه هیچ‌کدام نیازمند بازطراحی معماری نیستند:

جدول ۷-۱ - کارهای آینده به ترتیب اولویت

#	کار	شرح
۱	اجرای زمان‌بندی‌شده خزنده	تبدیل گردآوری یک‌باره نقطه‌ای به به‌روزرسانی دوره‌ای، با تشخیص تغییر و نگهداشتن نسخه‌های پیشین
۲	اعتبارسنجی میدانی مدل بازدید	مقایسه خروجی مدل با شمارش واقعی در چند نقطه نمونه و تنظیم ضرایب بر همان اساس
۳	آزمون مؤلفه و گسترش آزمون مرورگر	ده جریان مرورگر فعلی مسیرهای اصلی را می‌پوشانند؛ گام بعد پوشش همه صفحه‌ها و آزمون تک‌مؤلفه‌ای است
۴	مهاجرت به یک موتور کلاینت-سروری (مثل PostgreSQL)	یک فرمان، که کپی را انجام می‌دهد و شمارش دو طرف را تطبیق می‌دهد؛ آزموده و در دسترس. آنچه می‌ماند بازتولید تاریخچه مهاجرت‌ها روی موتور تازه است، چون تاریخچه فعلی به SQLite نوشته شده
۵	استقرار چندنمونه‌ای	کش و شمارنده‌های سقف نرخ هر دو با REDIS_URL مشترک می‌شوند و این مسیر با Redis روشن و خاموش آزموده شده؛ آنچه می‌ماند استقرار خود Redis و سنجش رفتار زیر بار است
۶	اتصال به درگاه پرداخت	پس از احراز شرایط حقوقی؛ منطق کسب‌وکار از پیش آماده است
۷	فعال‌سازی پیامک رمز یک‌بارمصرف	کل لایه پیامک و چرخه رمز یک‌بارمصرف پیاده‌سازی شده و آزمون خودکار دارد؛ فقط چون کلید سرویس تنظیم نشده در حالت خواب است و به محض افزودن KAVENEGAR_API_KEY به .env بدون هیچ تغییر کدی فعال می‌شود
۸	جمع‌آوری متمرکز رخدادها و هشدار	قالب ثبت رخداد از ابتدا ماشین‌خوان است، پس افزودن آن تغییری در کد لازم ندارد

۴-۷ جمع‌بندی

آنچه این پروژه را از یک تمرین دانشگاهی متمایز می‌کند، نه فهرست قابلیت‌هایش که **استدلال پشت مرزهایش** است. تصمیم به ساختن رزرو آنلاین، انتخاب یک موتور پایگاه داده ساده به همراه مستندکردن مسیر خروج از آن، پذیرش صریح اینکه محافظت از داده تماس مطلق نیست، و اصلاح مدلی که پیش‌تر عددی غیرواقعی تولید می‌کرد، همگی تصمیم‌هایی‌اند که ثبت و توجیه شده‌اند. سامانه‌ای که می‌داند چه کاری را انجام نمی‌دهد و چرا، از سامانه‌ای که همه چیز را ناقص ادعا می‌کند قابل‌انکاتر است.

فهرست مراجع

1. مستندات رسمی Next.js، نسخه ۱۶ – راهنمای App Router، مؤلفه‌های سمت سرور و لایه میانی. نسخه همراه بسته نصب‌شده در مخزن پروژه.
2. مستندات رسمی React، نسخه ۱۹ – مؤلفه‌های سمت سرور و مرز کلاینت.
3. مستندات Prisma ORM، نسخه ۷ – طرح‌واره، مهاجرت و لایه اتصال به موتور پایگاه داده.
4. مستندات SQLite – حالت WAL و رفتار همروندی.
5. OWASP Top Ten – فهرست ده ریسک برتر امنیت برنامه‌های تحت وب؛ مبنای بازبینی امنیتی این پروژه.
6. مستندات Zod – اعتبارسنجی طرح‌واره‌محور در TypeScript.
7. Google Maps – راهنمای جاسازی نقشه با iframe.
8. سرویس ژئوکدینگ نشان – مستندات واسط برنامه‌نویسی؛ برای یافتن مختصات جغرافیایی رکوردها.
9. Geopath – استاندارد سنجش مخاطب رسانه محیطی در ایالات متحده؛ مبنای شکل مدل تخمین بازدید.
10. Route Research Ltd. – استاندارد سنجش مخاطب رسانه محیطی در بریتانیا.
11. مرکز آمار ایران – نتایج سرشماری عمومی نفوس و مسکن؛ مبنای جمعیت شهرها در مدل.
12. شرکت کنترل ترافیک تهران – ارقام منتشرشده سرنشین به‌ارای خودرو در سفرهای شهری.
13. منابع داده رسانه محیطی به‌کاررفته در گردآوری: irbillboard.com, aradholding.com, billboardiha.com.
14. مستندات درون‌مخزنی پروژه: سند معماری، مرجع واسط برنامه‌نویسی، و سند تصمیم‌های مهندسی – در پوشه docs/ مخزن.

راهنمای اجرای پروژه

کد منبع

تمام کد، مستندات فنی و تاریخچه توسعه این پروژه در مخزن زیر در دسترس است:

github.com/alireza-zareian/Rasamap-Repository

۸-۱ پیش‌نیازها و راه‌اندازی

تنها پیش‌نیاز، محیط اجرای Node.js نسخه ۲۰ یا بالاتر است. هیچ سرویس بیرونی، پایگاه داده جداگانه یا کانتینری لازم نیست.

```
npm install
cp .env.example .env
# سپس مقادیر را پر کنید
npm run db:migrate && npm run db:seed
npm run demo
# ساخت + اجرا روی درگاه ۳۰۰۰
```

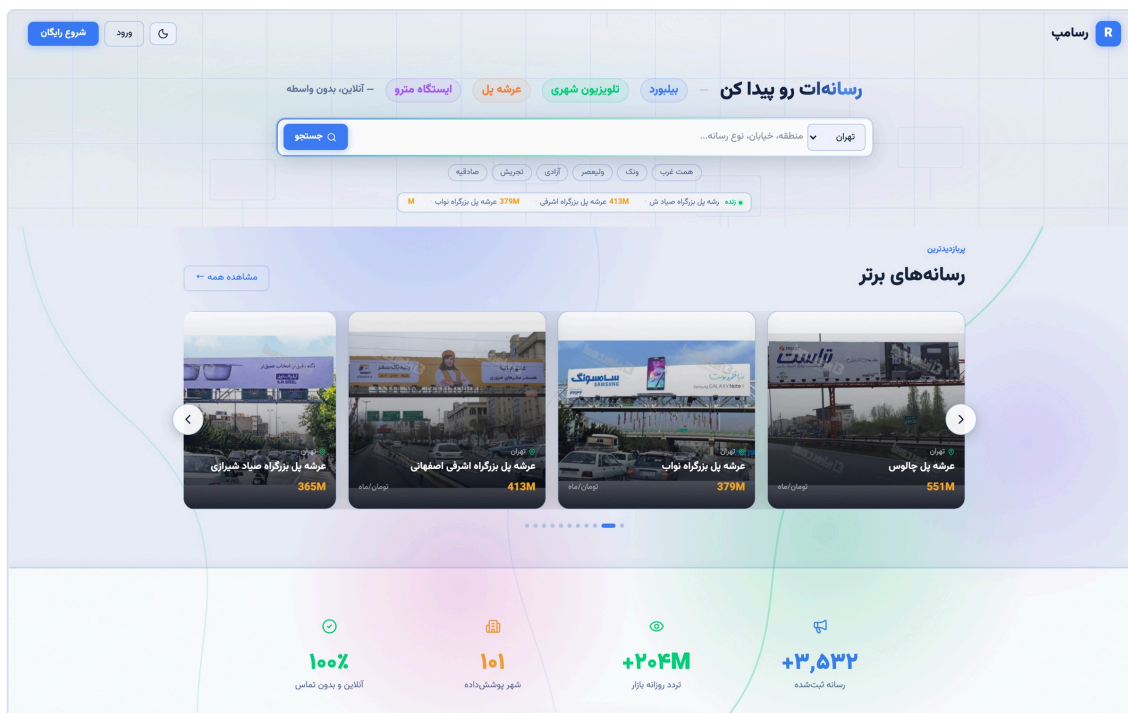
پس از اجرا، سامانه از هر دستگاه دیگری روی همان شبکه محلی با نشانی `http://<3000>:<3000>` در دسترس است؛ تمام رفتارهای وابسته به اتصال برای همین حالت آزموده شده‌اند (بخش ۲-۶). دستور `npm test` نیز ۱۷۹ آزمون را روی یک ساخت تولیدی و پایگاه داده ایزوله اجرا می‌کند (`npm run test:unit` فقط قاعده‌های خالص را، بدون ساخت، در نیم ثانیه)، و `npm run test:e2e` ۱۰ جریان کاربری را در مرورگر واقعی می‌راند.

قاعده عملیاتی – برای دیدن یا ارائه سامانه `npm run demo`

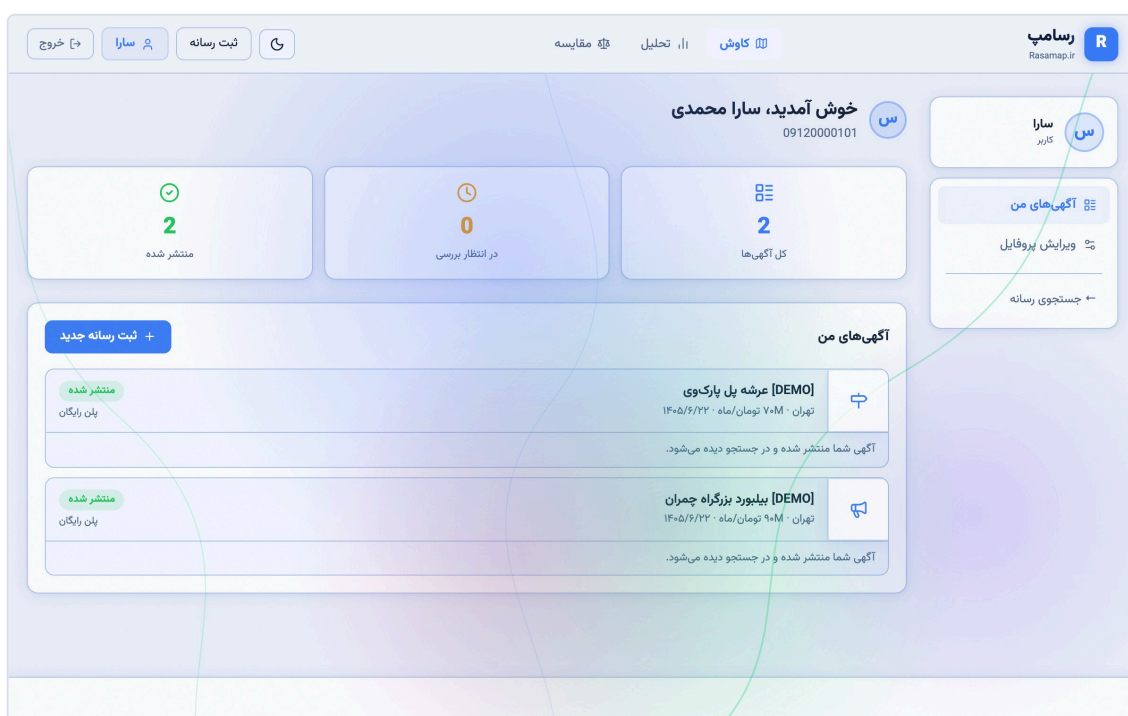
دستور `npm run dev` فقط هنگام نوشتن کد کاربرد دارد. برای نخستین بازدید از ده مسیر، حالت توسعه ۹/۷ ثانیه پردازنده مصرف می‌کند و حالت ساخته‌شده ۸/۰ ثانیه، یعنی حدود ۹۷ برابر کمتر.

نمایش از سامانه

تصاویر زیر از نسخه در حال اجرای سامانه گرفته شده‌اند و داده درونشان واقعی است. صفحه‌هایی که نیاز به ورود دارند با حساب‌های نمونه باز شده‌اند. صفحه جست‌وجو، صفحه جزئیات، فرم ثبت آگهی و صف بررسی در فصل‌های ۴ و ۵ آمده‌اند.

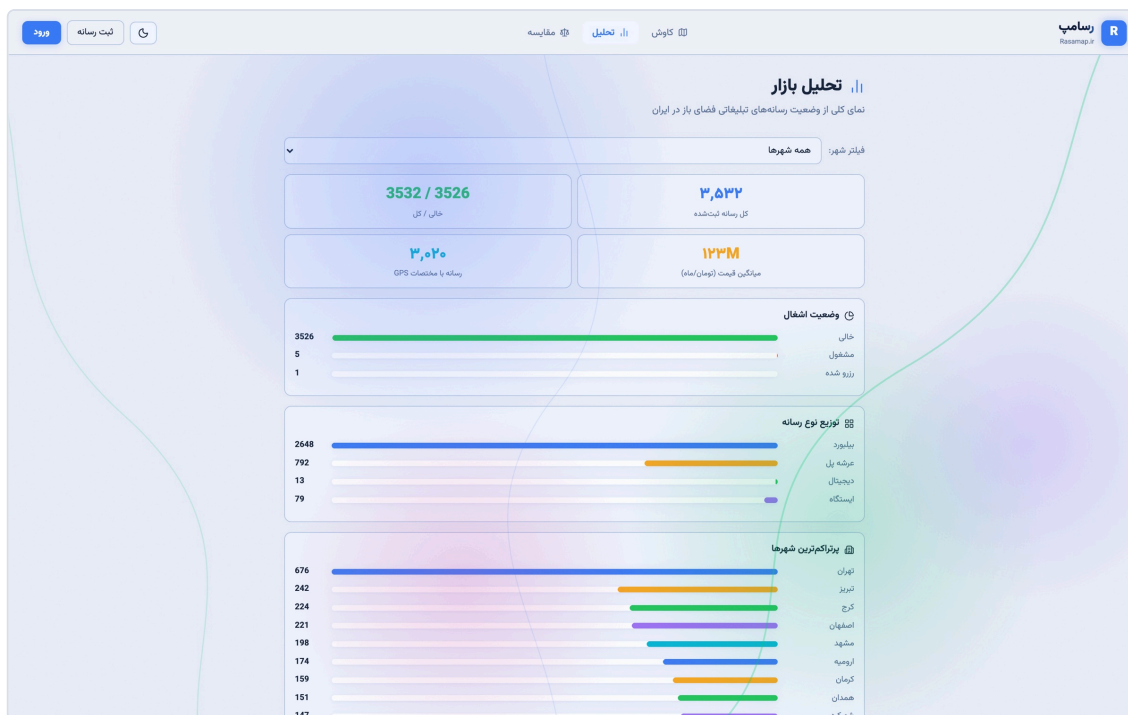


صفحه نخست



پیشخوان کاربر

شکل ۹-۱ - سمت کاربر: صفحه نخست با آمار کلی و رسانه‌های شاخص، و پیشخوانی که وضعیت هر آگهی ثبت‌شده را در چرخه تأیید نشان می‌دهد.



صفحهٔ تحلیل

مرجع واسط برنامه‌نویسی

< api.md در انتهای همین فایل ادغام شده است.

مرجع نقاط پایانی

Rasamap – HTTP API reference

All routes are Next.js Route Handlers under `app/api/`. Inputs are validated with `Zod .safeParse()`. User-facing error messages are in Persian. This document is maintained by hand – update it when a route changes

:This file is also rendered in-app at `/api-docs` (self-hosted, no external CDN). Demo accounts for trying the endpoints

[RUNBOOK.md](#)

Auth levels

	Meaning	Level
	no session needed	public
	<code>(rasamap_session) valid role: "user"</code> session cookie	user
session with role <code>viewer</code> / <code>editor</code> / <code>admin</code> / <code>super_admin</code> (enforced by <code>proxy.ts</code> and the route)		admin
<code>super_admin</code> role <code>editor</code> , <code>admin</code> or		+editor
<code>super_admin</code> role <code>admin</code> or		+admin

Conventions

- .Session is a signed JWT (jose, HS256) in an `HttpOnly SameSite=Strict` cookie
- .Admin route order is fixed: **session check** → **rate limit** → **Zod** → **business logic**
- .`( lib/auth/rate-limit.ts )` Rate limits are per-IP, in-memory sliding window
- .Auth failures use generic messages (no user enumeration)
- Idempotency-Key** (optional header on `POST /api/listings`)

مرجع واسط برنامه‌نویسی

شکل ۲-۹ – صفحهٔ تحلیل، توزیع شهر، نوع و قیمت را از دادهٔ واقعی رسم می‌کند. مرجع کامل هر ۴۴ نقطهٔ پایانی هم از دل خود سامانه ارائه می‌شود و به سنځ بیرونی نیازی نیست.

خروج

مشاهده سایت

ادمین [DEMO]

ادمین

پنل مدیریت

رسمایپ

مدیریت بیلورها

آیتم ۳,۵۳۶

جستجو...

همه انواع

همه وضعیت‌ها

همه (بررسی)

↑ ID

▼

نام / مکان	نوع	وضعیت	قیمت	مختصات	تصاویر	منبع	
بیلورد اتوبان همت غرب – تقاطع شیخ فضل‌الله تهران - اتوبان همت، قبل از تقاطع شیخ فضل‌الله.	بیلورد	رزرو شده	180M	ندارد	▲	manual	حذف ویرایش
تلویزیون شهری میدان ونک تهران - میدان ونک، ضلع شمالی، مقابل برج ونک	دیجیتال	خالی	320M	ندارد	▲	manual	حذف ویرایش
عرشه پل صدر – تقاطع مدرس تهران - تقاطع بزرگراه صدر و بزرگراه مدرس، عرشه	پل	مشغول	420M	ندارد	▲	manual	حذف ویرایش
بیلورد بزرگراه شیخ فضل‌الله تهران - بزرگراه شیخ فضل‌الله، کیلومتر ۳، سمت ج	بیلورد	خالی	95M	ندارد	▲	manual	حذف ویرایش
استراورد ایستگاه تجریش تهران - میدان تجریش، جنبه ایستگاه مترو، خروجی ش	ایستگاه	خالی	45M	ندارد	▲	manual	حذف ویرایش
بیلورد دیجیتال ولیعصر – میرداماد تهران - خیابان ولیعصر، تقاطع میرداماد، ضلع غرب	دیجیتال	خالی	285M	ندارد	▲	manual	حذف ویرایش
بیلورد آزادراه تهران-کرج تهران - آزادراه تهران-کرج، کیلومتر ۸، سمت تهرا	بیلورد	مشغول	240M	ندارد	▲	manual	حذف ویرایش
بیلورد بزرگراه رسالت – هنگام تهران - بزرگراه رسالت، بعد از خیابان هنگام، سم	بیلورد	خالی	130M	ندارد	▲	manual	حذف ویرایش
تلویزیون شهری پارک ملت	دیجیتال	مشغول	190M	ندارد	▲	manual	حذف ویرایش

نمای کلی

بیلورها

تأیید آگهی‌ها

سرنگ‌ها

کیفیت

اسکرپر

کاربران

لاگ

فهرست و ویرایش رسانه‌ها

خروج

مشاهده سایت

ادمین [DEMO]

ادمین

پنل مدیریت

رسمایپ

کنترل کیفیت

48 از 100 رکورد بررسی شده

این فهرست هر بار از روی همان داده‌های زندهٔ بیلورها ساخته می‌شود، جای ذخیره‌شده‌ای ندارد. بررسی روی 100 رکوردی انجام می‌شود که همین حالا در این پنل بارگذاری شده‌اند، نه روی کل جدول؛ شمارش کل رکوردهای بدون تصویر و بدون مختصات در تب «نمای کلی» است. یک رکورد وقتی «مورد کیفیت» می‌شود که یکی از این نبوده‌ها را داشته باشد: مختصات نداشتن، تصویر نداشتن، آدرس کوتاه‌تر از ۵ نویسه، قیمت کمتر از ۱، یا مختصاتی که بیرون محدودهٔ ایران بیفتد. شدت هم از روی شمار همین ایرادها تعیین می‌شود؛ سه ایراد یا بیشتر «بحرانی»، دو ایراد «متوسط»، یک ایراد «کم». برای اصلاح، روی «اصلاح رکورد» بزنید تا همان بیلورد در پنجرهٔ ویرایش باز شود؛ بعد از ذخیره، همین فهرست دوباره حساب می‌شود.

#1	بیلورد اتوبان همت غرب – تقاطع شیخ فضل‌الله	مختصات ندارد	تصویر ندارد	متوسط	اصلاح رکورد
#2	تلویزیون شهری میدان ونک	مختصات ندارد	تصویر ندارد	متوسط	اصلاح رکورد
#3	عرشه پل صدر – تقاطع مدرس	مختصات ندارد	تصویر ندارد	متوسط	اصلاح رکورد
#4	بیلورد بزرگراه شیخ فضل‌الله	مختصات ندارد	تصویر ندارد	متوسط	اصلاح رکورد

نمای کلی

بیلورها

تأیید آگهی‌ها

سرنگ‌ها

کیفیت

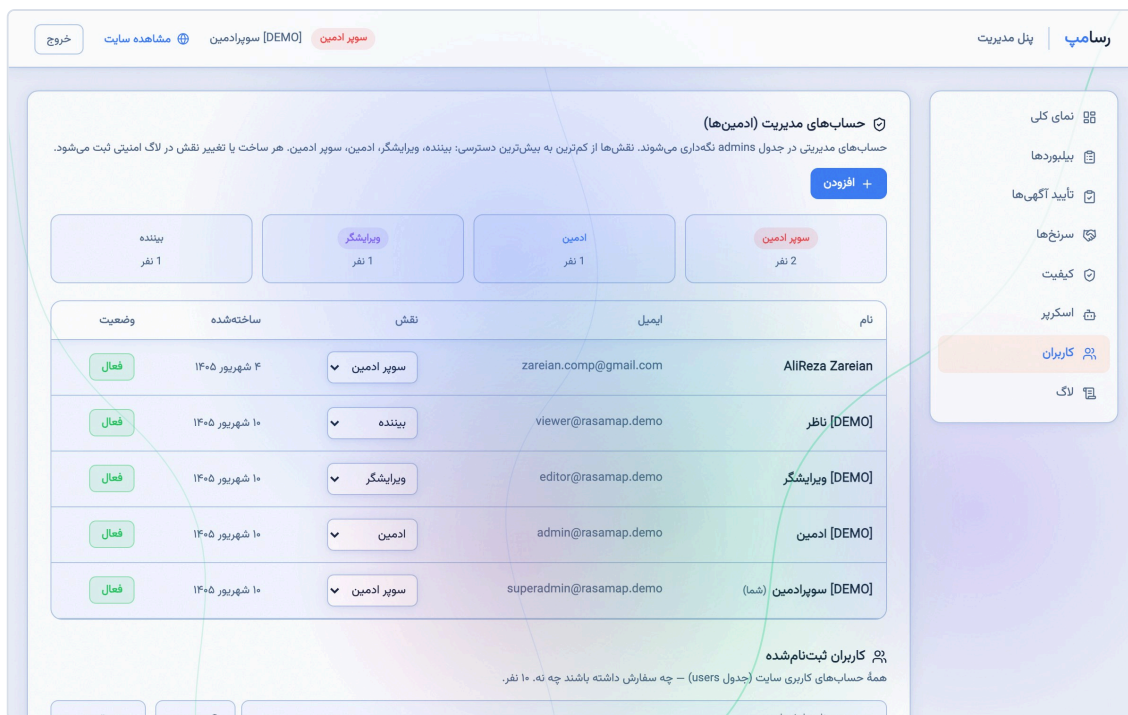
اسکرپر

کاربران

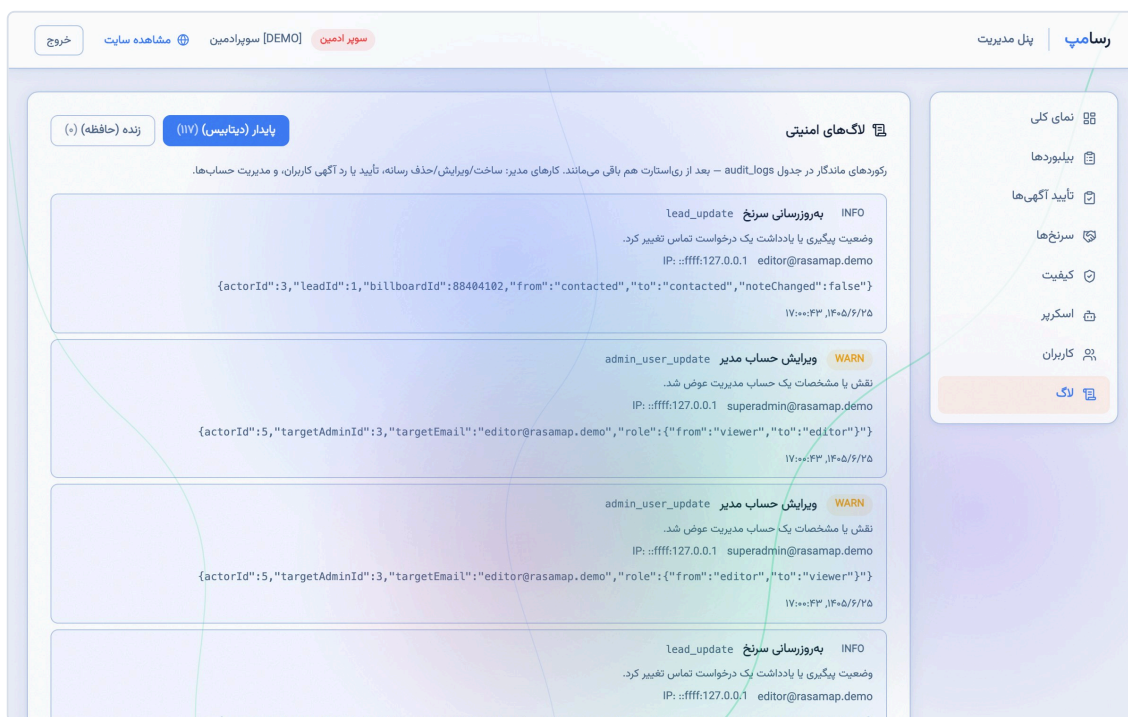
لاگ

وضعیت کیفیت داده

شکل ۳-۹ – پنل مدیریت: ویرایش رسانه‌ها، و صفحهٔ کیفیت که رکوردهای ناقص (بدون تصویر یا بدون مختصات) را برای پیگیری جدا می‌کند.



مدیریت مدیران و سطوح دسترسی



رد ممیزی

شکل ۴-۹ - چهار سطح دسترسی مدیران، و رد ممیزی که هر تغییر مدیریتی را با شناسه مدیر، نشانی شبکه و جزئیات تغییر نگه می‌دارد.

Abstract

The market for out-of-home advertising media in Iran (billboards, footbridge panels, transit shelters and urban displays) is fragmented. Each structure's details sit with its own owner or contractor, so a buyer looking for a suitable placement must contact several intermediaries separately. The result is that no single searchable view of the available supply exists.

This thesis presents the design and implementation of a web system that gathers that supply into one searchable catalogue. It serves two user groups: advertising buyers, who search and filter across 3,536 media units in 101 cities, view each unit's details and map position, and (after authenticating) obtain the owner's contact number; and media owners, who submit their own listings through a multi-step form for expert review before publication.

The system is implemented as a single codebase on Next.js 16 and React 19, with a data layer over SQLite through Prisma 7 and authentication by a signed token in an HttpOnly cookie. The interface is entirely Persian and right-to-left. Seed data was collected from three public sources by a Python scraper and consolidated after duplicate removal.

The work's distinguishing contribution is a daily-audience estimation model. None of the surveyed sources publishes an exposure figure, although this is precisely the number a buyer needs. The model follows the shape used by the international out-of-home industry — count passing traffic, convert it to people, then discount by the share who could actually see that particular face — driven by road class rather than city population, and is fully deterministic and reproducible.

Engineering effort concentrated on server-side quality: a single data access layer that is the only code allowed to reach the database (enforced by static analysis), one request pipeline written once and shared by every route, schema validation on all input, rate limiting keyed to a non-spoofable client identity, idempotency guarantees on non-idempotent operations, and a durable audit trail for every administrative change. Correctness is verified by 179 automated tests — 12 unit tests of the product rules, 160 API-level tests and 7 tests of the nightly data importer — and 10 end-to-end browser flows, all passing against a production build.

Keywords: out-of-home advertising, web marketplace, Next.js, server-side rendering, API security, concurrency, SQLite

University of Zanjan

Faculty of Engineering

Thesis Submitted in Partial Fulfilment of the Requirements
for the Degree of Bachelor of Science (B.Sc.) in Computer Engineering

Design and Implementation of a Billboard Search and Listing System

By

Alireza Zareian

Supervisor

Dr. Sajjad Haghzad Klidbary

September 2026